Università degli Studi di Bergamo

Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione

Modulo di Progettazione, Algoritmi e Computabilità (PAC)

AgenticUniBG

Assistente Virtuale Multi-Agente per l'Università degli Studi di Bergamo

Autori:
Luca Brugnetti – Matricola 1086670
Raphael Mazzoleni – Matricola 1086531
Michele Caprinali – Matricola 1087210

Anno Accademico 2025–2026

Indice

1 Introduzione **4**
1.1 Contesto del Progetto . . . 4
1.2 Obiettivi del Sistema . . . 4
1.3 Metodologia di Sviluppo: AMDD . . . 4
1.4 Stack Tecnologico . . . 5
1.5 Motivazioni delle Scelte . . . 5
1.5.1 Perché un'architettura multi-agente? . . . 5
1.5.2 Perché LangGraph? . . . 5
1.5.3 Perché Google Gemini? . . . 5
1.5.4 Perché MongoDB Atlas? . . . 6

2 Iterazione 0 – Visione Iniziale **7**
2.1 Obiettivo dell'Iterazione . . . 7
2.2 Analisi dei Requisiti e Use Case . . . 7
2.2.1 User Stories . . . 7
2.2.2 Use Cases . . . 9
2.2.3 Requisiti Non Funzionali . . . 9
2.2.4 MoSCoW . . . 9
2.3 Design dell'Architettura . . . 10
2.3.1 Diagramma degli Agenti . . . 10
2.3.2 Rischi Iniziali e Strategia di Evoluzione . . . 11

3 Iterazione 1 – MVP della Pipeline Multi-Agent **12**
3.1 Obiettivo dell'Iterazione . . . 12
3.2 Analisi dei Requisiti e Use Case . . . 12
3.2.1 User Stories . . . 12
3.2.2 Use Cases . . . 12
3.3 Design dell'Architettura . . . 13
3.3.1 Diagramma dei Componenti . . . 13
3.3.2 Diagramma delle Classi . . . 14
3.3.3 Diagramma di Sequenza . . . 15
3.3.4 Diagramma di Attività . . . 15
3.4 Scelte di Design . . . 16
3.4.1 AgentState come TypedDict . . . 16
3.4.2 Groq LLAMA 70B come LLM . . . 16
3.4.3 Pipeline sequenziale . . . 16
3.4.4 Separazione Frontend/Backend . . . 17
3.5 Analisi Dinamica . . . 17

4 Iterazione 2 – Autenticazione e Profilazione **18**
4.1 Obiettivo dell'Iterazione . . . 18
4.2 Analisi dei Requisiti e Use Case . . . 18
4.2.1 User Stories . . . 18

4.2.2 Use Cases . . . 19
4.2.3 Requisiti Non Funzionali . . . 21
4.3 Design dell'Architettura . . . 22
4.3.1 Diagramma dei Componenti . . . 22
4.3.2 Diagramma delle Classi . . . 24
4.3.3 Diagramma di Sequenza . . . 25
4.3.4 Diagramma di Attività . . . 26
4.4 Scelte di Design . . . 26
4.4.1 AgentState con Campi di Profilazione . . . 26
4.4.2 Containerizzazione con Docker Compose . . . 27
4.5 Analisi Dinamica . . . 27

5 Iterazione 3 – JWT, Gestione Sessione e Memoria Conversazionale 28
5.1 Obiettivo dell'Iterazione . . . 28
5.2 Analisi dei Requisiti e Use Case . . . 29
5.2.1 Use Cases . . . 30
5.3 Design dell'Architettura per la Memoria Conversazionale . . . 30
5.3.1 Opzione 1 – `localStorage` lato frontend . . . 30
5.3.2 Opzione 2 – Redis lato server + persistenza su MongoDB . . . 31
5.3.3 Opzione 3 – Salvataggio turn-by-turn solo su MongoDB . . . 32
5.3.4 Decisione Finale . . . 33
5.4 Design dell'Architettura . . . 34
5.4.1 Diagramma delle Classi . . . 34
5.4.2 Diagramma di Sequenza . . . 36
5.4.3 Diagramma di Attività . . . 37
5.5 Analisi Dinamica . . . 37

6 Iterazione 4 – Web Search, Query Agent e Logger 39
6.1 Obiettivo dell'Iterazione . . . 39
6.2 Analisi dei Requisiti e Use Case . . . 39
6.2.1 Use Cases . . . 39
6.2.2 Requisiti Non Funzionali . . . 40
6.3 Design dell'Architettura . . . 41
6.3.1 Diagramma dei Componenti . . . 41
6.3.2 Diagramma delle Classi . . . 43
6.3.3 Diagramma di Sequenza . . . 44
6.3.4 Diagramma di Attività . . . 45
6.4 Analisi Dinamica . . . 45

7 Iterazione 5 – Gestione Profilo, Timestamp Temporale, Date degli Esami e Prompt Engineering 47
7.1 Obiettivo dell'Iterazione . . . 47
7.2 Analisi dei Requisiti e Use Case . . . 48
7.2.1 User Stories . . . 48
7.2.2 Use Cases . . . 48
7.2.3 Requisiti Non Funzionali . . . 51
7.3 Design dell'Architettura . . . 51
7.3.1 Diagramma dei Componenti . . . 51
7.3.2 Diagramma delle Classi . . . 53

7.3.3 Diagramma di Sequenza . . . 55
7.4 Scelte di Design . . . 56
7.4.1 Routing Condizionale `date_esami` con Selezione LLM del Calendario 56
7.4.2 SettingsPage con Dizionario `DIPARTIMENTI_CORSI` . . . 57
7.4.3 Timestamp tramite `get_italian_timestamp()` . . . 57
7.4.4 History come Messaggi LLM Nativi . . . 57
7.4.5 Prompt Engineering Iterativo . . . 57
7.4.6 Timing nel Logger senza Overhead . . . 58
7.5 Analisi Dinamica . . . 58

8 Design in Piccolo 59
8.1 Algoritmo 1 — Esecuzione del Workflow come visita di un DAG . . . 59
8.1.1 Contesto . . . 59
8.1.2 Pseudocodice . . . 59
8.1.3 Analisi di complessità temporale . . . 60
8.2 Algoritmo 2 — Ordinamento e selezione Top-K dei risultati web . . . 61
8.2.1 Contesto . . . 61
8.2.2 Pseudocodice . . . 61
8.2.3 Analisi di complessità temporale . . . 61

9 Analisi Statica 63
9.1 Strumenti Utilizzati . . . 63
9.1.1 Radon . . . 63
9.1.2 SonarQube for IDE . . . 63
9.2 Metriche Raw (Radon) . . . 63
9.3 Complessità Ciclomatica (Radon) . . . 64
9.3.1 Risultati per Classe . . . 65
9.4 Analisi SonarQube for IDE . . . 65
9.4.1 Riepilogo delle Issue . . . 66
9.4.2 Issue di Naming Convention . . . 66
9.4.3 Issue sulla Cognitive Complexity . . . 66

10 Quality Evaluation basata su Test 68
10.1 Valutazione Complessiva della Qualità . . . 68
10.2 Prestazioni per Domanda . . . 70
10.3 Analisi per Esperto . . . 71
10.4 Inter-Expert Agreement . . . 71
10.4.1 Analisi del Kappa di Cohen . . . 72
10.4.2 Analisi del Kappa di Fleiss . . . 72
10.4.3 Interpretazione e Validità . . . 72
10.5 Risultati Principali e Considerazioni . . . 73

11 Conclusioni e Sviluppi Futuri 74
11.1 Riepilogo delle Iterazioni . . . 74
11.2 Obiettivi Raggiunti . . . 74
11.3 Sviluppi Futuri . . . 75

Capitolo 1

Introduzione

1.1 Contesto del Progetto

Il sito web dell'Università degli Studi di Bergamo presenta una struttura gerarchica complessa, con informazioni distribuite su numerose pagine e documenti. Gli studenti devono navigare manualmente tra molte sezioni o consultare documenti lunghi per ricercare una singola informazione, con conseguenti perdite di tempo.

Agentic UniBG nasce con l'obiettivo di sviluppare un sistema intelligente multi-agent in grado di assistere studenti e utenti nella navigazione del sito universitario, fornendo risposte rapide, contestualizzate e personalizzate a qualsiasi domanda pertinente, riducendo notevolmente il tempo di ricerca.

1.2 Obiettivi del Sistema

Il sistema è concepito come un assistente conversazionale avanzato, capace di:

- Comprendere richieste in linguaggio naturale
- Recuperare informazioni reali e aggiornate dal sito dell'ateneo
- Generare risposte strutturate, accurate e contestualizzate
- Personalizzare l'esperienza in base al profilo dello studente
- Gestire conversazioni con memoria del contesto

1.3 Metodologia di Sviluppo: AMDD

Il progetto adotta la metodologia **Agile Model Driven Development (AMDD)**, che combina i principi dello sviluppo agile con la modellazione iterativa e incrementale. Per supportare questa modalità abbiamo utilizzato GitHub per la gestione della repository (https://github.com/RaphacePrime/AgenticUniBG) e Discord, per poter parlare e confrontrarsi tra i membri del team sullo sviluppo del progetto.

L'approccio AMDD prevede:

1. **Iterazione 0 – Envisioning**: definizione della visione iniziale, dei requisiti ad alto livello e dell'architettura target. Nessun codice prodotto.
2. **Iterazioni successive**: sviluppo incrementale con cicli brevi, ciascuno dei quali produce un incremento funzionante del sistema. I modelli UML vengono creati e aggiornati ad ogni iterazione secondo necessità.

3. **Evoluzione continua**: i requisiti, l'architettura e la documentazione evolvono di pari passo con il codice.

1.4 Stack Tecnologico

Layer	Tecnologia	Motivazione
Frontend	React	SPA reattiva
Backend	Python 3.11 + FastAPI	Framework asincrono e semplice per endpoint API
Orchestrazione AI	LangGraph + LangChain	Framework per grafi di agenti LLM
Modello LLM	Google Gemini	LLM utilizzato come motore per i nostri agenti (API key non gratuita)
Ricerca Web	Tavily API	Motore di ricerca ottimizzato per agenti AI
Database	MongoDB Atlas	Database noSQL
Autenticazione	JWT + bcrypt	Token stateless, hashing sicuro
Containerizzazione	Docker + Docker Compose	Deployment riproducibile e isolato

Tabella 1.1: Stack tecnologico del progetto

1.5 Motivazioni delle Scelte

1.5.1 Perché un'architettura multi-agente?

Un singolo prompt monolitico risulta fragile: mescola classificazione, generazione e formattazione in un'unica chiamata LLM. Separando le responsabilità in agenti distinti è possibile ispezionare l'output di ogni fase, l'aggiunta di nuovi agenti è semplificata in quanto si inseriscono come nuovi nodi del grafo e la qualità del tono e della struttura migliora consistentemente.

1.5.2 Perché LangGraph?

LangGraph permette di definire pipeline di agenti come grafi orientati, con stato condiviso tipizzato. Rispetto a una catena sequenziale semplice, LangGraph supporta branching condizionale e cicli, preparando il sistema a evoluzioni future o inserimento di nuovi agenti.

1.5.3 Perché Google Gemini?

Google Gemini è stato scelto come provider LLM per i seguenti motivi:

- **Qualità del modello**: Gemini offre capacità di ragionamento avanzate, particolarmente efficaci nella comprensione del contesto.
- **Integrazione con LangChain**: il pacchetto fornisce un'integrazione nativa e ben mantenuta con l'ecosistema LangChain.

- **Latenze competitive**: le API Gemini offrono tempi di risposta contenuti, permettendo l'esecuzione di più chiamate sequenziali mantenendo tempi di risposta accettabili per l'utente.

1.5.4 Perché MongoDB Atlas?

Lo schema del profilo utente è semplice ma potenzialmente variabile. MongoDB permette flessibilità nello schema senza migrazioni. Atlas fornisce hosting gestito con backup automatico.

Capitolo 2

Iterazione 0 – Visione Iniziale

2.1 Obiettivo dell'Iterazione

L'Iterazione 0 rappresenta la fase di *envisioning* prevista dalla metodologia AMDD. Non è stato prodotto codice funzionante, ma è stata definita la visione complessiva del progetto, i requisiti funzionali e non funzionali ad alto livello e le user stories iniziali, seguendo le idee proposte dai membri del team.

2.2 Analisi dei Requisiti e Use Case

2.2.1 User Stories

UC-01 – Login come Studente

Descrizione: Lo studente accede al sistema inserendo le proprie credenziali. Il sistema verifica le credenziali e, in caso di successo, autentica l'utente e lo reindirizza alla schermata principale. Se lo studente non possiede un account, viene incluso il caso d'uso UC-02 (Registrazione Studente).

UC-02 – Registrazione Studente

Descrizione: Lo studente compila un modulo di registrazione fornendo i propri dati personali. Il sistema verifica la validità dei dati, crea un nuovo profilo nel database e conferma l'avvenuta registrazione (non deve essere già presente nel sistema).

UC-03 – Gestione Profilo Utente

Descrizione: Lo studente può visualizzare e modificare le informazioni del proprio profilo. Il sistema aggiorna i dati nel database e conferma le modifiche.

UC-04 – Accesso come Ospite

Descrizione: L'utente accede al sistema senza autenticazione. Il sistema consente l'accesso alle funzionalità generali, senza personalizzazione delle risposte.

UC-05 – Inviare Domanda/Richiesta

Descrizione: L'utente inserisce una domanda tramite l'interfaccia conversazionale. Il sistema riceve il messaggio, lo inoltra all'Orchestrator Agent che coordina gli agenti specializzati per elaborare la risposta. A seconda del contenuto della richiesta, il sistema può attivare uno dei sotto-casi d'uso specifici (UC-06 – UC-11). Al termine dell'elaborazione, viene incluso il caso d'uso UC-12 (Ricevere Risposta).

UC-06 – Richiedere Guida Passo-Passo

Descrizione: Il sistema riconosce che la richiesta riguarda una procedura e fornisce una guida strutturata passo-passo, indicando i passaggi necessari, i documenti richiesti e i link utili.

UC-07 – Richiedere Documenti

Descrizione: Il sistema riconosce che la richiesta riguarda un documento ufficiale e ne recupera il contenuto o il link diretto dalla base di conoscenza interna, restituendolo all'utente.

UC-08 – Informazioni Universitarie Generali

Descrizione: Il sistema riconosce che la richiesta riguarda informazioni generali sull'ateneo e recupera le informazioni pertinenti dalla base di conoscenza o tramite ricerca web, restituendo una risposta strutturata.

UC-09 – Informazioni sui Corsi

Descrizione: Il sistema riconosce che la richiesta riguarda un corso di studio e recupera le informazioni aggiornate dal database o dalla base di conoscenza, restituendo i dettagli richiesti.

UC-10 – Consultare Occupazione Aule

Descrizione: Il sistema riconosce che la richiesta riguarda la verifica della disponibilità di un'aula in un determinato momento. Restituisce lo stato dell'aula con eventuali dettagli sugli orari disponibili.

UC-11 – Consultare Orari Lezioni

Descrizione: Il sistema riconosce che la richiesta riguarda l'orario di una o più lezioni. Recupera i dati aggiornati dalla fonte dati appropriata e restituisce l'orario richiesto, indicando giorno, ora, aula e docente.

UC-12 – Ricevere Risposta

Descrizione: Il sistema restituisce la risposta generata all'utente tramite l'interfaccia conversazionale. La risposta è formattata in modo chiaro e strutturato, includendo eventuali link, riferimenti a documenti o indicazioni procedurali. L'utente può opzionalmente fornire un feedback sulla risposta ricevuta (UC-13).

UC-13 – Fornire Feedback sulla Risposta

Descrizione: L'utente può opzionalmente fornire un feedback sulla qualità e la pertinenza della risposta. Il sistema registra il feedback per migliorare le risposte future.

2.2.2 Use Cases

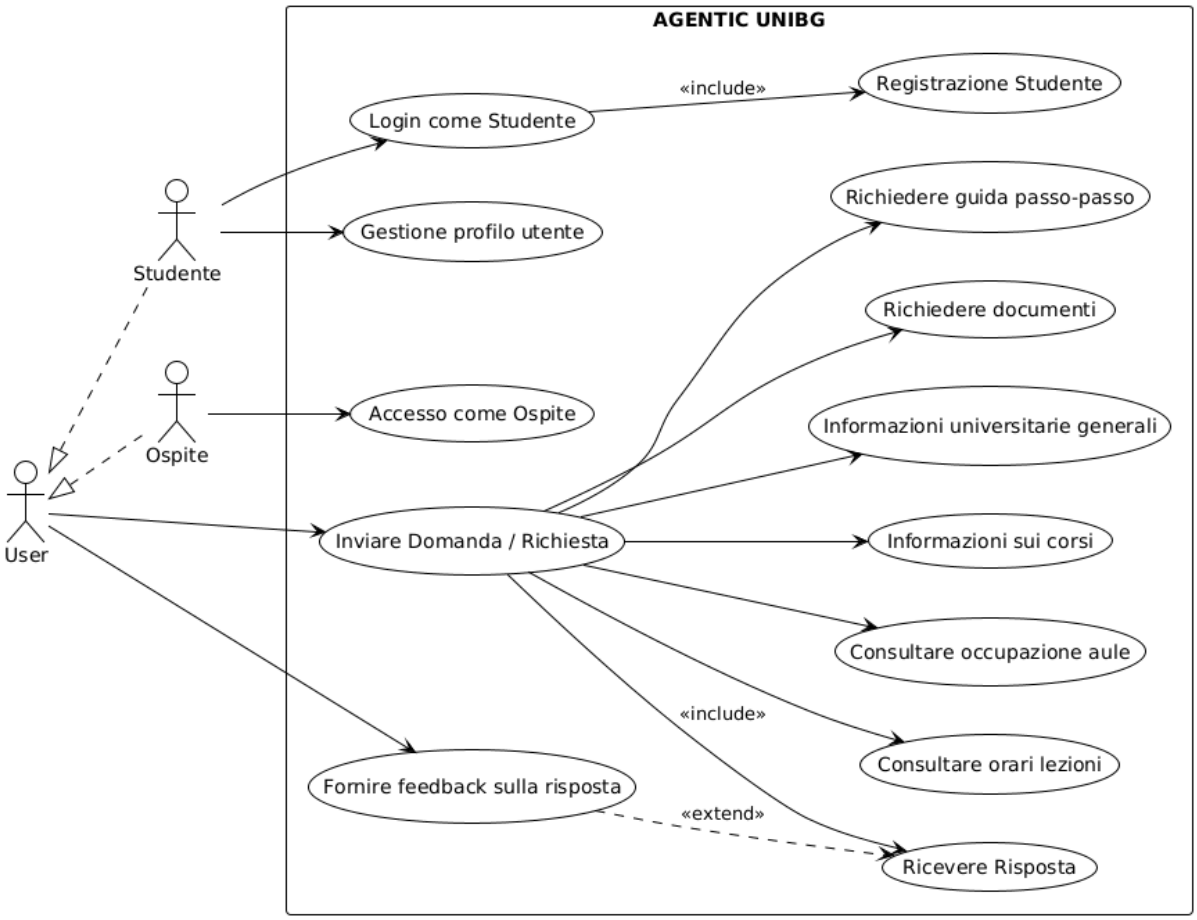

Figura 2.1: Use case diagram

2.2.3 Requisiti Non Funzionali

- Architettura modulare e scalabile
- Estensibilità degli agenti
- Separazione delle responsabilità
- Tempo di risposta accettabile
- Supporto utenti autenticati e ospiti
- Persistenza dei dati

2.2.4 MoSCoW

Abbiamo deciso di assegnare una priorità, basandoci sul modello MoSCoW, ad ogni use case così da facilitare la scelta dello sviluppo nelle iterazioni successive.

ID	Caso d'Uso	Priorità MoSCoW
UC-01	Login come Studente	Must Have
UC-02	Registrazione Studente	Must Have
UC-03	Gestione Profilo Utente	Should Have
UC-04	Accesso come Ospite	Must Have
UC-05	Inviare Domanda / Richiesta	Must Have
UC-06	Richiedere Guida Passo-Passo	Must Have
UC-07	Richiedere Documenti	Should Have
UC-08	Informazioni Universitarie Generali	Must Have
UC-09	Informazioni sui Corsi	Should Have
UC-10	Consultare Occupazione Aule	Could Have
UC-11	Consultare Orari Lezioni	Could Have
UC-12	Ricevere Risposta	Must Have
UC-13	Fornire Feedback sulla Risposta	Won't Have

Tabella 2.1: Prioritizzazione dei casi d'uso

2.3 Design dell'Architettura

L'architettura target prevede tre macro-componenti:

1. **Frontend**: applicazione React per l'interazione utente;
2. **Backend**: API Gateway, Orchestrator Agent e agenti specializzati;
3. **Persistence Layer**: database per profili studente e cronologia conversazioni;
4. **Componenti esterni**: LLM Provider, accesso web.

2.3.1 Diagramma degli Agenti

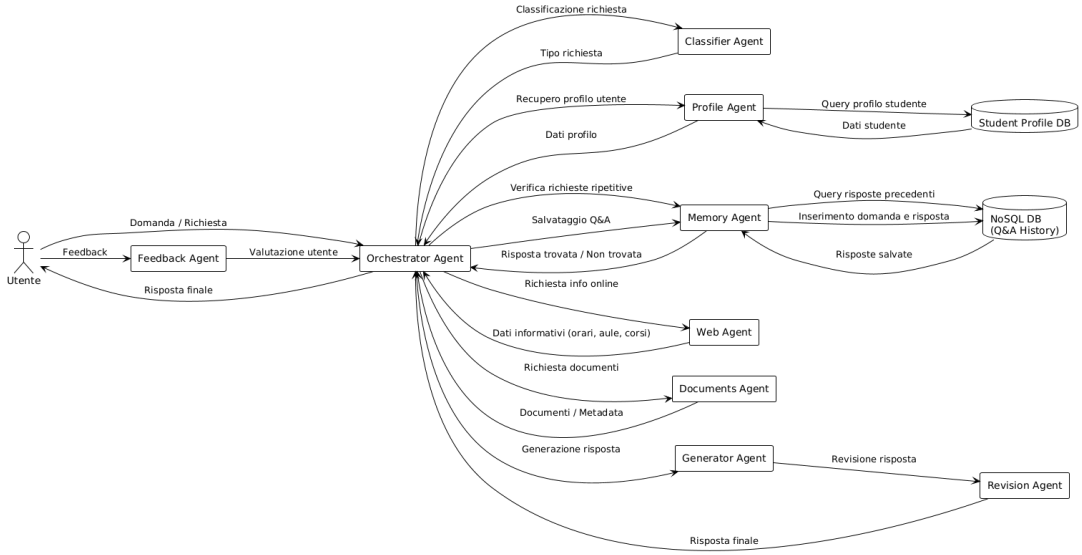

Figura 2.2: Diagramma degli agenti previsti

Il flusso teorico completo prevede:

1. Il Frontend invia la richiesta all'API Gateway via REST.
2. L'API Gateway la inoltra all'Orchestrator Agent.
3. L'Orchestrator instrada la richiesta agli agenti specializzati nell'ordine opportuno: **Classifier Agent** → classifica l'intento, **Profile Agent** → recupera il profilo dal database SQL, **Memory Agent** → recupera la cronologia dal database NoSQL Q&A History, **Web Agent** → ricerca su internet, **Documents Agent** → ricerca su documenti interni, **Generator Agent** → genera la risposta chiamando l'LLM Provider, **Revision Agent** → revisiona la bozza, **Feedback Agent** → raccoglie il feedback dell'utente per migliorare le risposte successive.
4. La risposta finale risale all'Orchestrator e viene restituita al Frontend.

2.3.2 Rischi Iniziali e Strategia di Evoluzione

Data la complessità dell'idea iniziale, il team ha deciso di implementare inizialmente un "Minimum Viable Product" ed ad ogni iterazione evolvere l'architettura progressivamente, aggiornando i modelli UML ad ogni iterazione. Queste scelte sono dovuete alla complessità architetturale elevata, alla dipendenza da servizi LLM esterni e l'incertezza sui tempi e qualità delle risposte generate.

Capitolo 3

Iterazione 1 – MVP della Pipeline Multi-Agent

3.1 Obiettivo dell'Iterazione

Implementare il primo Minimum Viable Product (MVP) funzionante, limitato alla pipeline conversazionale di base: ricezione della domanda, classificazione dell'intento, generazione della risposta e revisione. L'architettura è organizzata in **due tier** (Client + Application Logic).

3.2 Analisi dei Requisiti e Use Case

3.2.1 User Stories

ID	Descrizione
US-01	Come studente, voglio inviare una domanda e ricevere una risposta
US-02	Come sistema, voglio classificare la richiesta per generare una risposta coerente
US-03	Come sistema, voglio generare una risposta tramite LLM pertinente alla domanda
US-04	Come sistema, voglio verificare la qualità della risposta per ridurre errori

Tabella 3.1: User stories – Iterazione 1

3.2.2 Use Cases

- **UC-I1 – Invio Richiesta**: l'utente invia una domanda. Se la query è vuota il frontend blocca l'invio; se il backend fallisce viene mostrato un errore generico.
- **UC-I2 – Classificazione Intento**: il ClassifierAgent invia un prompt strutturato all'LLM con le categorie disponibili. Se la categoria restituita non è valida, fallback a `altro`.
- **UC-I3 – Generazione Risposta**: il GeneratorAgent costruisce un prompt contestualizzato alla categoria e invoca l'LLM.
- **UC-I4 – Revisione Risposta**: il RevisionAgent revisiona la bozza. In caso di errore, la bozza originale viene usata come fallback.

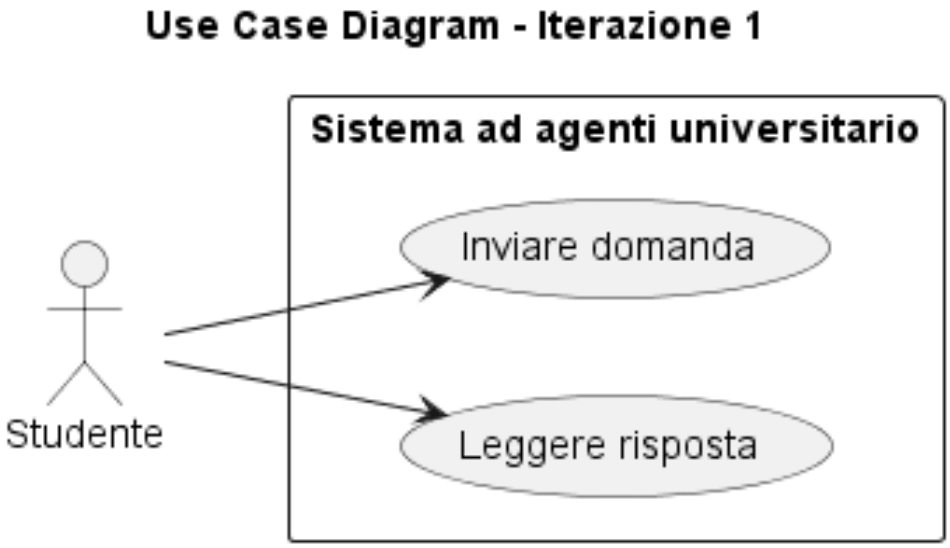

Figura 3.1: Diagramma dei casi d'uso – Iterazione 1

Il diagramma dei casi d'uso mostra un singolo attore (Studente) con due casi d'uso: "Inviare domanda" e "Leggere risposta".

3.3 Design dell'Architettura

3.3.1 Diagramma dei Componenti

I componenti sviluppati sono:

- **OrchestratorAgent**: costruisce il grafo LangGraph con tre nodi (`classify` → `generate` → `revise`)
- **ClassifierAgent**: classifica la query in una delle categorie
- **GeneratorAgent**: genera la risposta con prompt per categoria
- **RevisionAgent**: revisiona la bozza
- **AgentState**: `TypedDict` condiviso per lo stato della pipeline

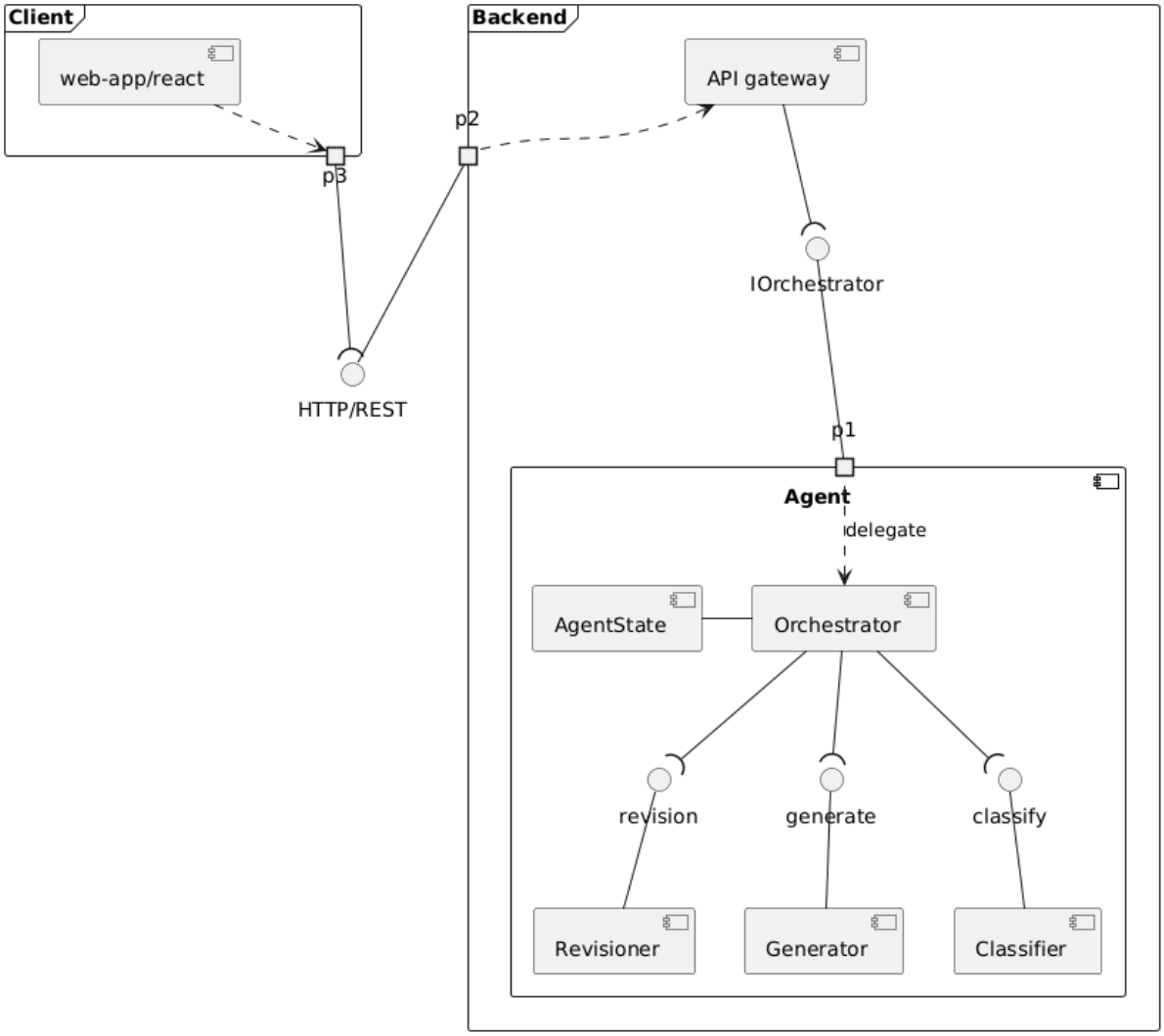

Figura 3.2: Diagramma dei componenti – Iterazione 1

3.3.2 Diagramma delle Classi

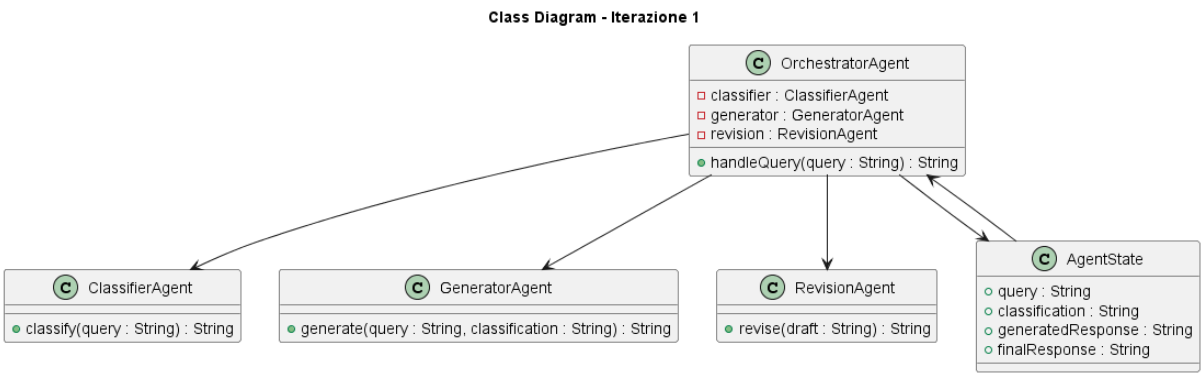

Figura 3.3: Diagramma delle classi – Iterazione 1

Il diagramma delle classi modella l'AgentState (stato condiviso della pipeline), il ClassifierAgent, GeneratorAgent, RevisionAgent e l'OrchestratorAgent.

3.3.3 Diagramma di Sequenza

Il diagramma di sequenza illustra il flusso completo di una query: dall'inserimento della domanda da parte dell'utente fino alla visualizzazione della risposta.

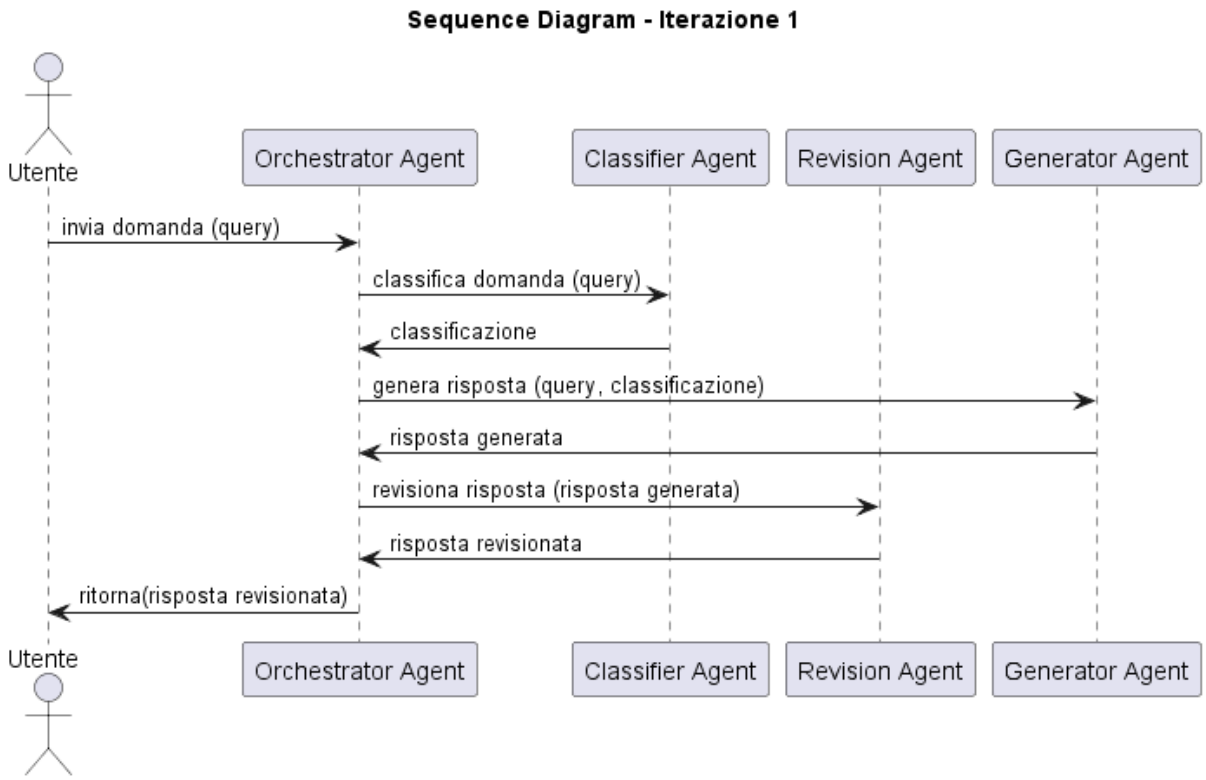

Figura 3.4: Diagramma di sequenza – Pipeline classify → generate → revise

3.3.4 Diagramma di Attività

Il diagramma di attività mostra il workflow della pipeline: ricezione query → classificazione → generazione → revisione → risposta.

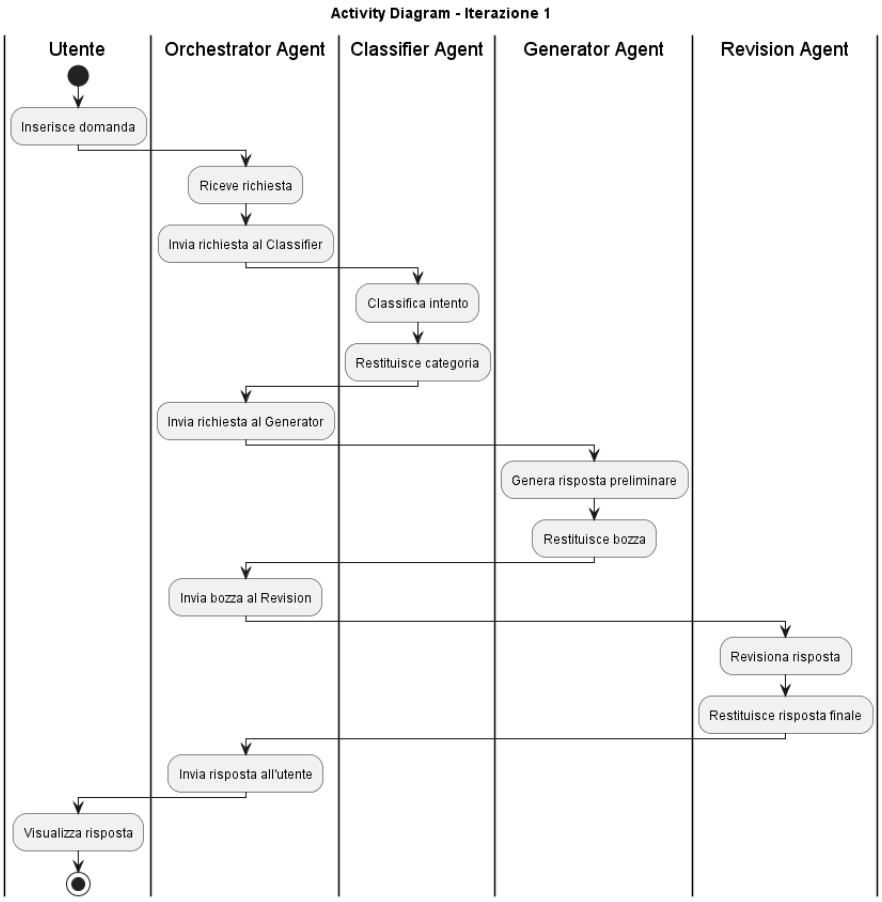

Figura 3.5: Diagramma di attività – Pipeline Iterazione 1

3.4 Scelte di Design

3.4.1 AgentState come TypedDict

Si è scelto TypedDict perché LangGraph richiede uno stato serializzabile come dizionario. TypedDict fornisce type-checking statico senza overhead runtime, utile per chiamate API.

3.4.2 Groq LLAMA 70B come LLM

Tutti gli agenti utilizzano il modello OLLAMA 70B tramite l'integrazione. La scelta di Groq garantisce:

- Risposte di qualità in lingua italiana.
- Latenze contenute per l'esecuzione sequenziale di più agenti.
- Un tier gratuito sufficiente per le fasi di sviluppo e testing.

3.4.3 Pipeline sequenziale

Nella prima iterazione la pipeline è strettamente sequenziale (classify → generate → revise). Questa scelta semplifica il debugging ed è sufficiente per il MVP. L'uso di

LangGraph permette di evolvere verso pipeline più complesse nelle iterazioni successive senza refactoring strutturale.

3.4.4 Separazione Frontend/Backend

Sin dalla prima iterazione, frontend e backend sono progetti separati con Dockerfile distinti. Questa scelta prepara il deployment in container isolati e permette sviluppo parallelo.

3.5 Analisi Dinamica

L'analisi dinamica del sistema è stata condotta attraverso test API eseguiti con Postman. I test verificano il corretto funzionamento di tutti gli endpoint REST esposti dal backend FastAPI, organizzati per iterazione di sviluppo.

Le variabili utilizzate per poter effettuare le chiamate API sono:

1. base-url: http://localhost:8000 (per potersi connettere in locale al server).
2. test-matricola: 1234567.
3. test-passoword: test2026.

ID	Endpoint	Metodo	Descrizione	Risultato PostMan
IT1-01	`/api/health`	GET	Controllo funzionamento sistema	200 OK
IT1-02	`/api/agent/query`	POST	Query generica ospite	200 OK
IT1-03	`/api/agent/query`	POST	Query su docente	200 OK
IT1-04	`/api/agent/query`	POST	Query vuota	Errore

Tabella 3.2: Test API – Iterazione 1

Capitolo 4

Iterazione 2 – Autenticazione e Profilazione

4.1 Obiettivo dell'Iterazione

L'Iterazione 2 evolve il prototipo funzionale dell'Iterazione 1 verso un'applicazione completa e deployabile, introducendo tre macro-funzionalità:

- **Autenticazione utente**: registrazione e login tramite matricola e password, con hashing bcrypt e persistenza su MongoDB Atlas.
- **Profilazione e personalizzazione**: il profilo dello studente (corso, dipartimento, anno, tipologia) viene recuperato dopo il login e iniettato come contesto nei prompt degli agenti, producendo risposte contestualizzate.
- **Infrastruttura completa**: frontend React, containerizzazione Docker Compose, layer di autenticazione dedicato con pattern Controller → Service → Repository.

L'architettura evolve da **due tier** (Iterazione 1) a **tre tier**: Client, Backend e Data Tier (MongoDB Atlas).

4.2 Analisi dei Requisiti e Use Case

4.2.1 User Stories

ID	Descrizione	Ruolo
US-05	Come studente, voglio registrarmi fornendo matricola e dati accademici, per accedere al sistema con il mio profilo	Studente
US-06	Come studente, voglio autenticarmi e ricevere il mio profilo, così il sistema può personalizzare le risposte	Studente
US-07	Come ospite, voglio inviare domande senza autenticazione, ottenendo risposte generiche	Ospite
US-08	Come studente autenticato, voglio ricevere risposte contestuali al mio corso e anno di frequentazione	Studente
US-09	Come utente, voglio inviare una richiesta testuale e ricevere una risposta elaborata dall'intera pipeline ad agenti	Utente

Tabella 4.1: User stories – Iterazione 2

4.2.2 Use Cases

- **UC-01 – Registrazione Studente**

  Descrizione: Lo studente compila il form di registrazione (nome, cognome, matricola, password, dipartimento, corso, tipologia, anno). Il backend verifica l'unicità della matricola, hasha la password con bcrypt e persiste il documento su MongoDB. Il profilo viene restituito al frontend.

- **UC-02 – Login Studente**

  Descrizione: Lo studente inserisce matricola e password nel form di login. Il backend cerca il documento tramite matricola, esegue `bcrypt.checkpw` e, in caso di successo, restituisce il profilo completo. Il frontend salva il profilo in memoria e abilita la chat.

- **UC-03 – Accesso come Ospite**

  Descrizione: L'utente seleziona la modalità ospite dal frontend. Non viene effettuata alcuna chiamata di autenticazione.

- **UC-04 – Inviare Richiesta**

  Descrizione: L'utente digita la domanda nell'interfaccia chat. Il frontend invia il campo `query` e, se disponibile, `user_info`. L'`OrchestratorAgent` avvia la pipeline di elaborazione.

- **UC-05 – Ricevere Risposta (Pipeline)**

  Descrizione: L'`OrchestratorAgent` inizializza l'`AgentState` con query e profilo utente. Il `ClassifierAgent` determina la categoria dell'intento. Il `GeneratorAgent` produce una risposta contestualizzata al profilo. Il `RevisionAgent` raffina la bozza. La risposta finale viene restituita tramite l'API Gateway.

- **UC-06 – Recupero Profilo**

  Descrizione: `AuthService` delega il recupero a `ProfileRepository.findById(matricola)`, che interroga MongoDB. Il documento viene restituito dopo aver rimosso il campo `passwordHash`.

- **UC-07 – Risposta Personalizzata**

  Descrizione: L'`OrchestratorAgent` invoca `build_user_context(state)` con i campi del profilo (nome, corso, anno, dipartimento, tipologia). La stringa di contesto risultante viene iniettata nel prompt del `ClassifierAgent` e del `GeneratorAgent`, orientando il modello verso informazioni rilevanti per il percorso specifico dello studente.

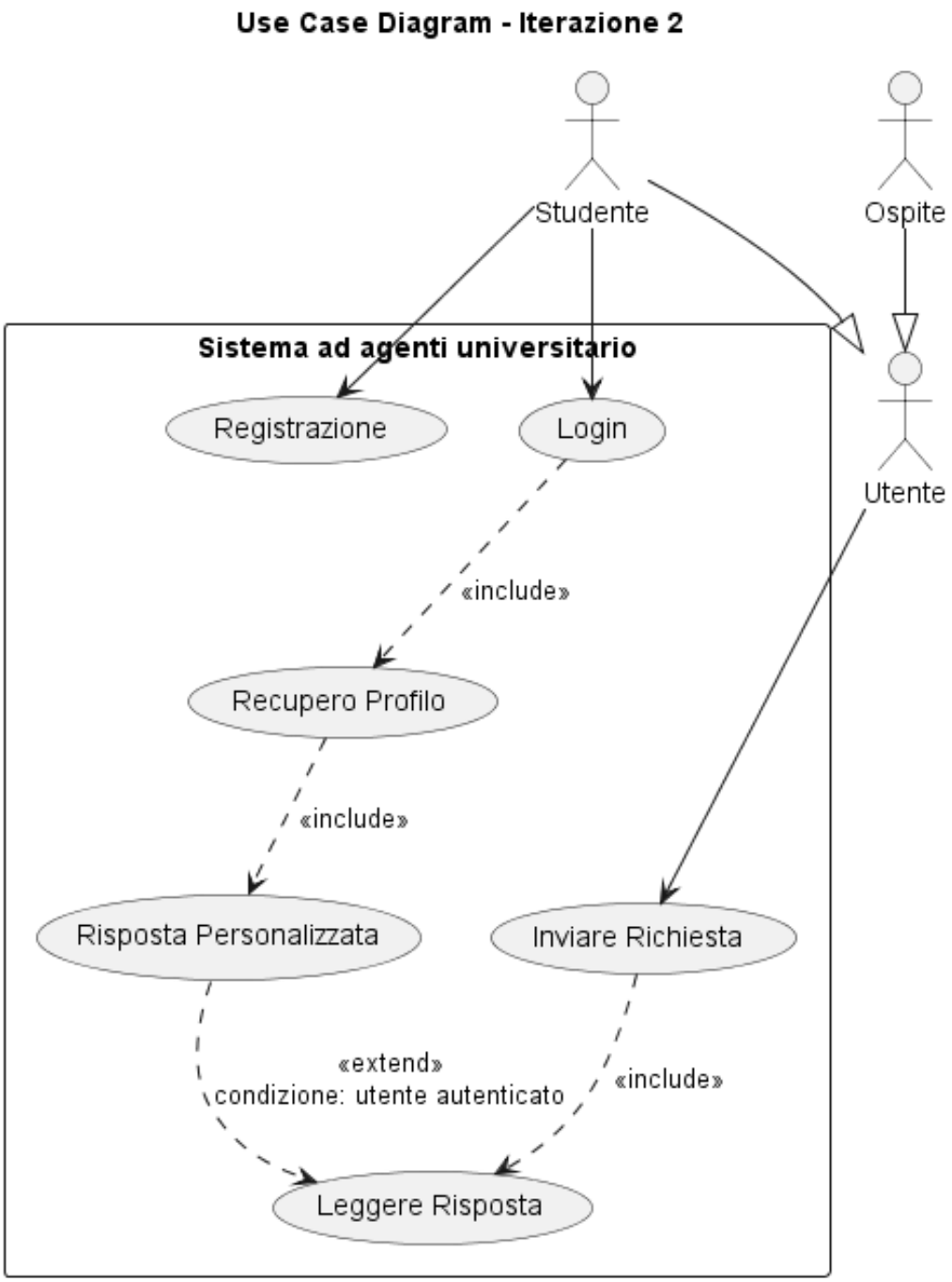

Figura 4.1: Diagramma dei casi d'uso – Iterazione 2

Il diagramma evidenzia due attori distinti: lo **Studente** (autenticato, con accesso a Registrazione, Login e Risposta Personalizzata) e l'**Ospite** (accesso diretto alla chat senza profilazione). Entrambi convergono sull'attore generalizzato **Utente** per il caso d'uso "Inviare Richiesta", che include sempre "Leggere Risposta". La Risposta Personalizzata *estende* "Leggere Risposta" con la condizione "utente autenticato", sottolineando la natura opzionale della personalizzazione.

4.2.3 Requisiti Non Funzionali

- **RNF-1 – Sicurezza password**: le password sono salvate esclusivamente come hash bcrypt; non trasmesse nelle risposte API.
- **RNF-2 – Persistenza**: i profili sono persistiti su MongoDB Atlas.
- **RNF-3 – Separazione dati sensibili**: il campo `passwordHash` non viene mai incluso nelle risposte API; il metodo `_public_profile` filtra i campi riservati prima di ogni risposta.
- **RNF-4 – Containerizzazione**: il sistema è deployabile tramite `docker-compose up` senza configurazione manuale.

4.3 Design dell'Architettura

4.3.1 Diagramma dei Componenti

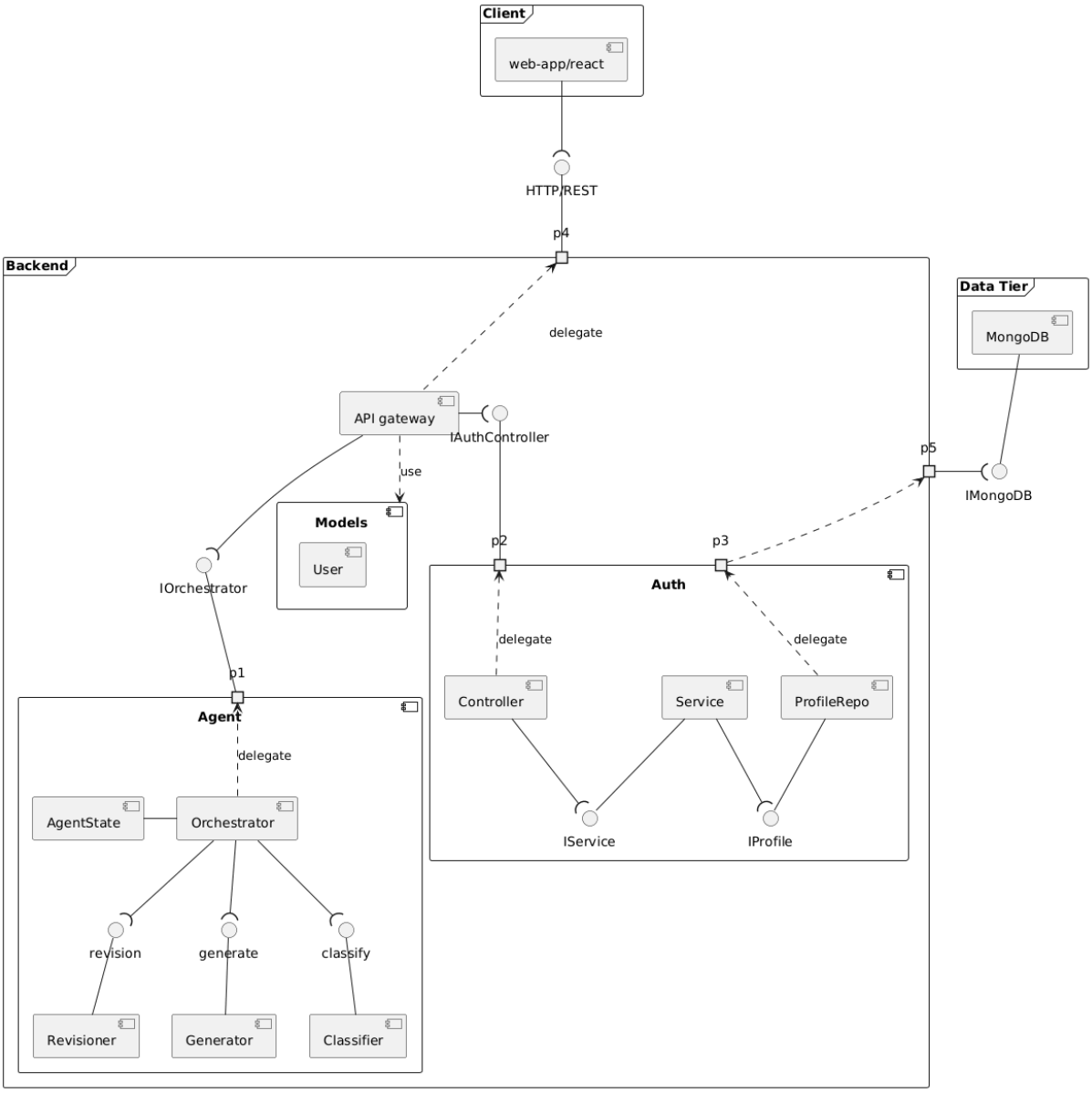

Figura 4.2: Diagramma dei componenti – Architettura a tre tier, overview (Iterazione 2)

L'architettura è articolata in tre tier comunicanti tramite interfacce definite:

- **Client Tier**: la Web App (React + Vite) espone l'interfaccia utente e comunica con il Backend tramite `HTTP/REST` sulla porta `pHttpIn`.
- **Backend Tier**: l'API Gateway (FastAPI) indirizza le richieste verso il layer agenti (tramite interfaccia `IOrchestrator`) o verso il layer di autenticazione (tramite interfaccia `IAuthAPI`). Il layer Auth interagisce con il Data Tier attraverso la `pDataOut1`/`IProfileRepo`.

- **Data Tier**: MongoDB Atlas, raggiunto dal `ProfileRepository` tramite interfaccia `IMongoDriver`.

Il diagramma dettagliato evidenzia i tre sottosistemi interni al Backend: il sottosistema **Agent** (AgentState, Orchestrator, Revisor, Generator, Classifier), il sottosistema **Auth** (Controller, Service, ProfileRepo con le interfacce `IService` e `IProfile`) e i **Models** condivisi (User). L'API Gateway orchestra entrambi i sottosistemi tramite le rispettive interfacce.

4.3.2 Diagramma delle Classi

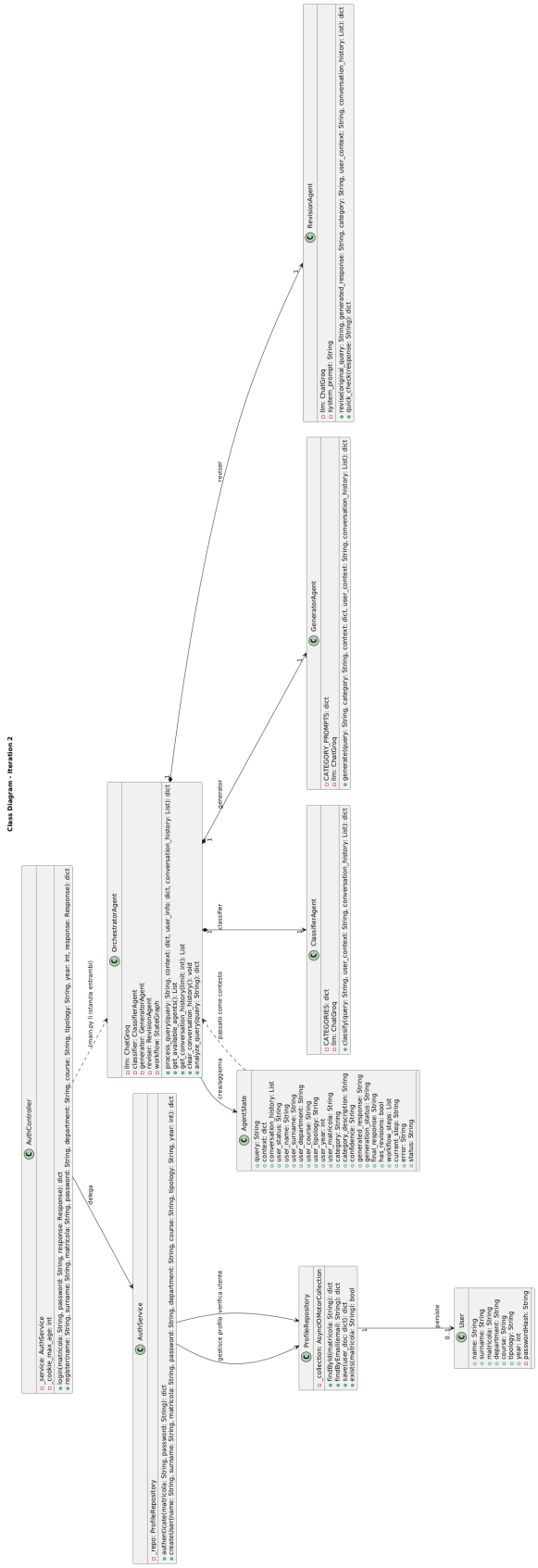

Figura 4.3: Diagramma delle classi – Iterazione 2

Il diagramma mostra due layer distinti interconnessi dall'API Gateway (`main.py`), che istanzia sia `AuthController` che `OrchestratorAgent`:

Layer Autenticazione :

- `AuthController`: espone gli endpoint HTTP `/login` e `/register`; filtra i campi sensibili prima della risposta tramite `_public_profile`; delega tutta la logica ad `AuthService`.
- `AuthService`: implementa `authenticate(matricola, password)` con verifica bcrypt e `createUser(...)` con hashing; interagisce direttamente con `ProfileRepository`.
- `ProfileRepository`: incapsula le operazioni CRUD su MongoDB (`findById`, `save`, `exists`).

Layer Agenti :

- `AgentState` acquisisce i campi di profilazione (`user_status`, `user_name`, `user_surname`, `user_course`, `user_department`, `user_tipology`, `user_year`, `user_matricola`) e il metodo `build_user_context`.
- `OrchestratorAgent` riceve il profilo come parametro in `process_query` e lo propaga nello stato.
- `ClassifierAgent` e `GeneratorAgent` ricevono il contesto utente come parametro nei rispettivi metodi.

4.3.3 Diagramma di Sequenza

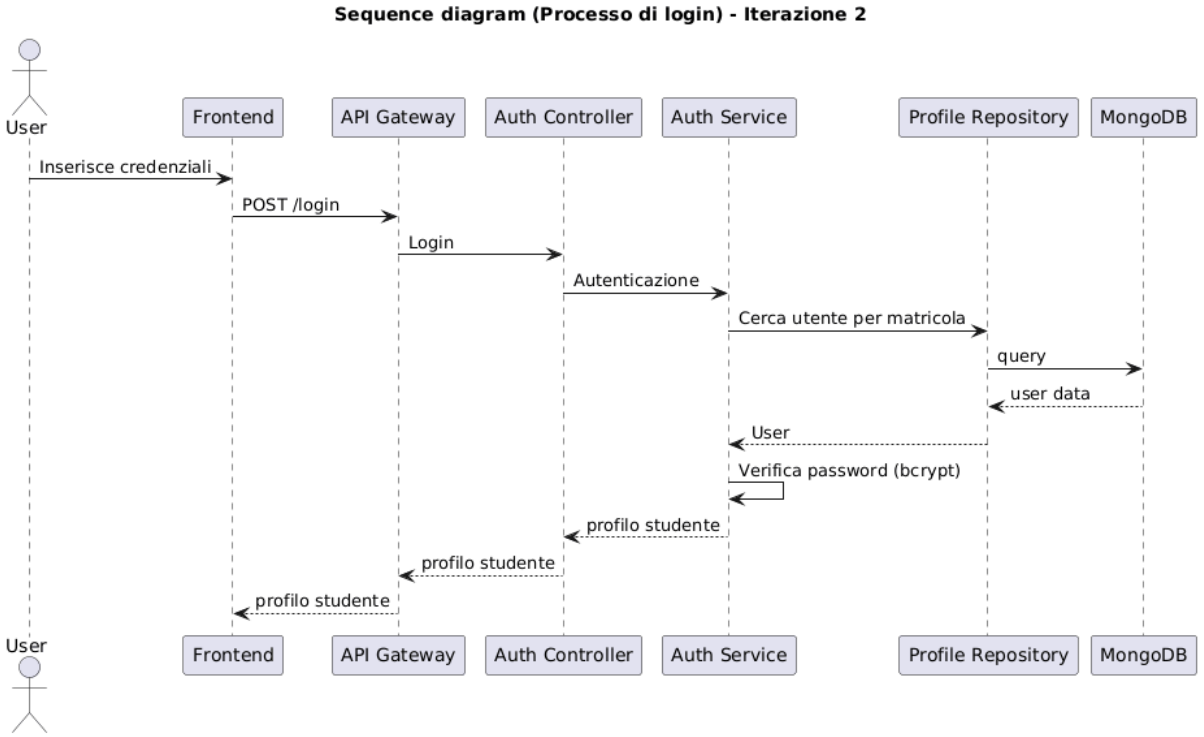

Figura 4.4: Diagramma di sequenza – Processo di login (Iterazione 2)

4.3.4 Diagramma di Attività

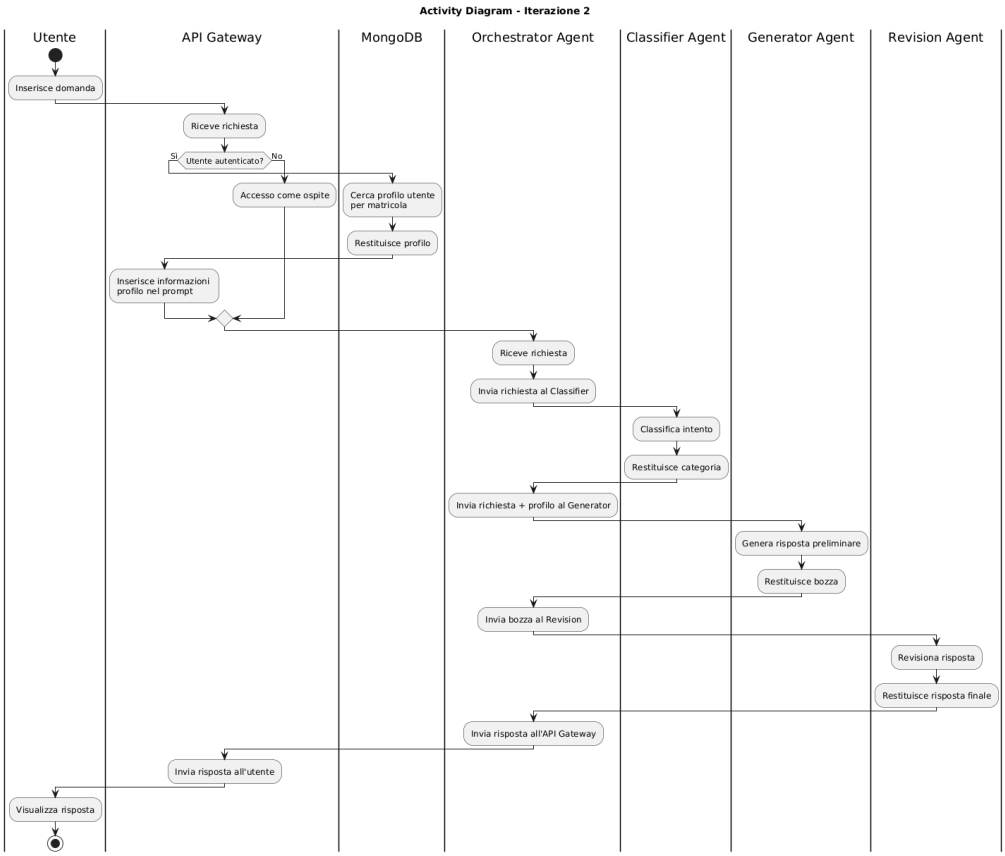

Figura 4.5: Diagramma di attività – Flusso di autenticazione e query personalizzata (Iterazione 2)

La decisione chiave è il nodo **"Utente autenticato?"** all'ingresso dell'API Gateway:

- **Sì**: il gateway interroga MongoDB per recuperare il profilo e lo inietta nel prompt tramite `build_user_context`.
- **No (Ospite)**: il gateway inserisce direttamente un contesto generico nel prompt senza accedere al database.

In entrambi i casi il flusso converge sull'Orchestrator, che avvia la pipeline classify → generate → revise e restituisce la risposta all'utente.

4.4 Scelte di Design

4.4.1 AgentState con Campi di Profilazione

L'estensione dell'`AgentState` con i campi del profilo è la scelta minimale per introdurre la personalizzazione senza modificare la firma dei metodi degli agenti. La funzione `build_user_context(state)` centralizza la costruzione del contesto testuale, rendendola modificabile in un unico punto. Gli agenti ricevono una stringa formattata pronta all'uso senza dipendere dalla struttura del profilo.

4.4.2 Containerizzazione con Docker Compose

Sin dall'Iterazione 2, il deploy prevede due container distinti (frontend Nginx + backend uvicorn) orchestrati da Docker Compose. Questa scelta garantisce riproducibilità dell'ambiente di sviluppo e produzione, isola le dipendenze e prepara l'architettura alla scalabilità orizzontale.

4.5 Analisi Dinamica

L'analisi dinamica è condotta tramite test API eseguiti con Postman, estendendo la collection dell'Iterazione 1. Il riferimento a tali test è presente nel file postman-collection.json. Le variabili utilizzate sono sempre le stesse della precedente iterazione.

ID	Endpoint	Met.	Descrizione	Atteso
IT2-01	`/api/auth/register`	POST	Registrazione nuovo studente	200 OK
IT2-02	`/api/auth/register`	POST	Matricola già registrata	409 CONFLICT
IT2-03	`/api/auth/login`	POST	Login con credenziali corrette	200 OK
IT2-04	`/api/auth/login`	POST	Matricola inesistente	401 UNAUTHORIZED
IT2-05	`/api/auth/login`	POST	Password errata	401 UNAUTHORIZED
IT2-06	`/api/agent/query`	POST	Query personalizzata con accesso	200
IT2-07	`/api/agents`	POST	Logout	200 OK

Tabella 4.2: Test API – Iterazione 2

Capitolo 5

Iterazione 3 – JWT, Gestione Sessione e Memoria Conversazionale

5.1 Obiettivo dell'Iterazione

L'obiettivo dell'iterazione 3 è consolidare il sistema con tre macro-funzionalità:

- **JWT Authentication via httpOnly Cookie**: aggiunta di un token di sessione e di accesso, salvato su cookie, questo permette di mantenere l'accesso alla chat in modo sicuro.
- **Gestione della Sessione Browser**: ripristino automatico al ricaricamento tramite `GET /api/auth/verify`
- **Memoria Conversazionale**: aggiunta di una memoria conversazionale che permette agli agenti di rispondere avendo un contesto della conversazione.

5.2 Analisi dei Requisiti e Use Case

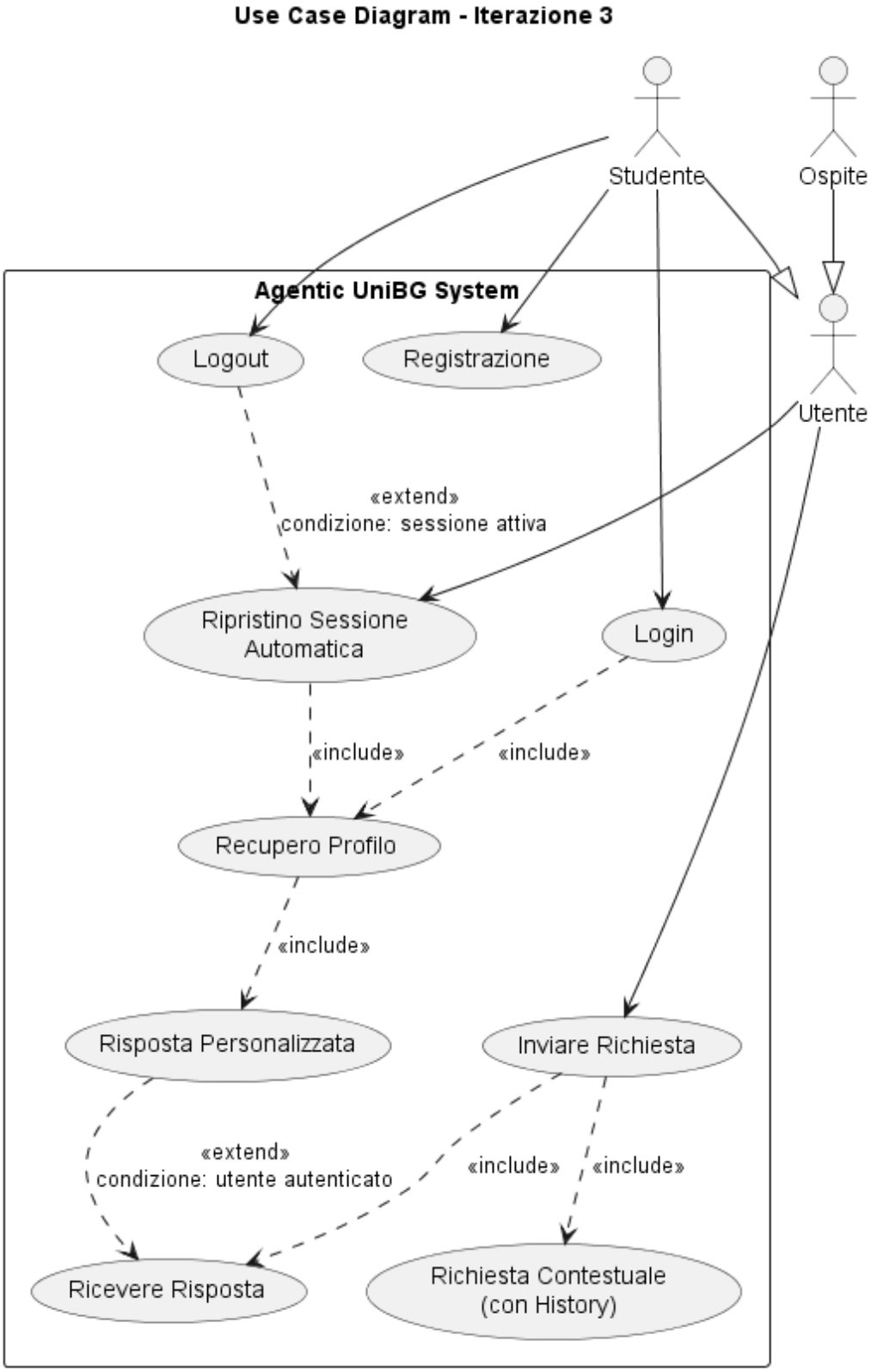

Figura 5.1: Diagramma dei casi d'uso – Iterazione 3

5.2.1 Use Cases

- **UC-08 – Registrazione** (aggiornato): al termine, `JWTManager.generateToken()` genera il token e `AuthController._set_auth_cookie()` imposta l'httpOnly cookie.
- **UC-09 – Login** (aggiornato): `AuthService.authenticate()` verifica bcrypt → `JWTManager.generateToken()` → cookie impostato → frontend carica conversazione da localStorage.
- **UC-10 – Ripristino Sessione Automatica**: al caricamento React, `GET /api/auth/verify` con `credentials: "include"` → backend decodifica JWT dal cookie → lookup MongoDB → profilo restituito → conversazione da localStorage. Flusso alternativo: token assente/scaduto → pagina login.
- **UC-11 – Logout Sicuro**: `POST /api/auth/logout` → `response.delete_cookie("access_toke` → frontend rimuove profilo e localStorage → redirect login.
- **UC-12 – Richiesta Contestuale**: il frontend include gli ultimi 10 messaggi (5 turni) nel body.

5.3 Design dell'Architettura per la Memoria Conversazionale

Per l'implementazione della memoria conversazionale, abbiamo valutato tre opzioni architetturali alternative con obiettivi diversi: semplicità operativa, performance, scalabilità e persistenza storica.

5.3.1 Opzione 1 – `localStorage` lato frontend

In questa opzione la memoria conversazionale viene mantenuta nel browser dell'utente (`localStorage`). Ad ogni richiesta il frontend invia al backend anche la `conversation_history` (ultimi turni), così gli agenti possono usare il contesto senza mantenere stato server-side.

Vantaggi:

- Implementazione semplice e veloce, senza nuovi componenti infrastrutturali
- Nessun costo operativo aggiuntivo
- Bassa latenza lato server: il backend resta stateless rispetto alla chat

Svantaggi:

- Memoria legata al dispositivo/browser (non condivisa tra device)
- Storico non centralizzato lato backend
- Possibile perdita dello storico in caso di pulizia storage browser

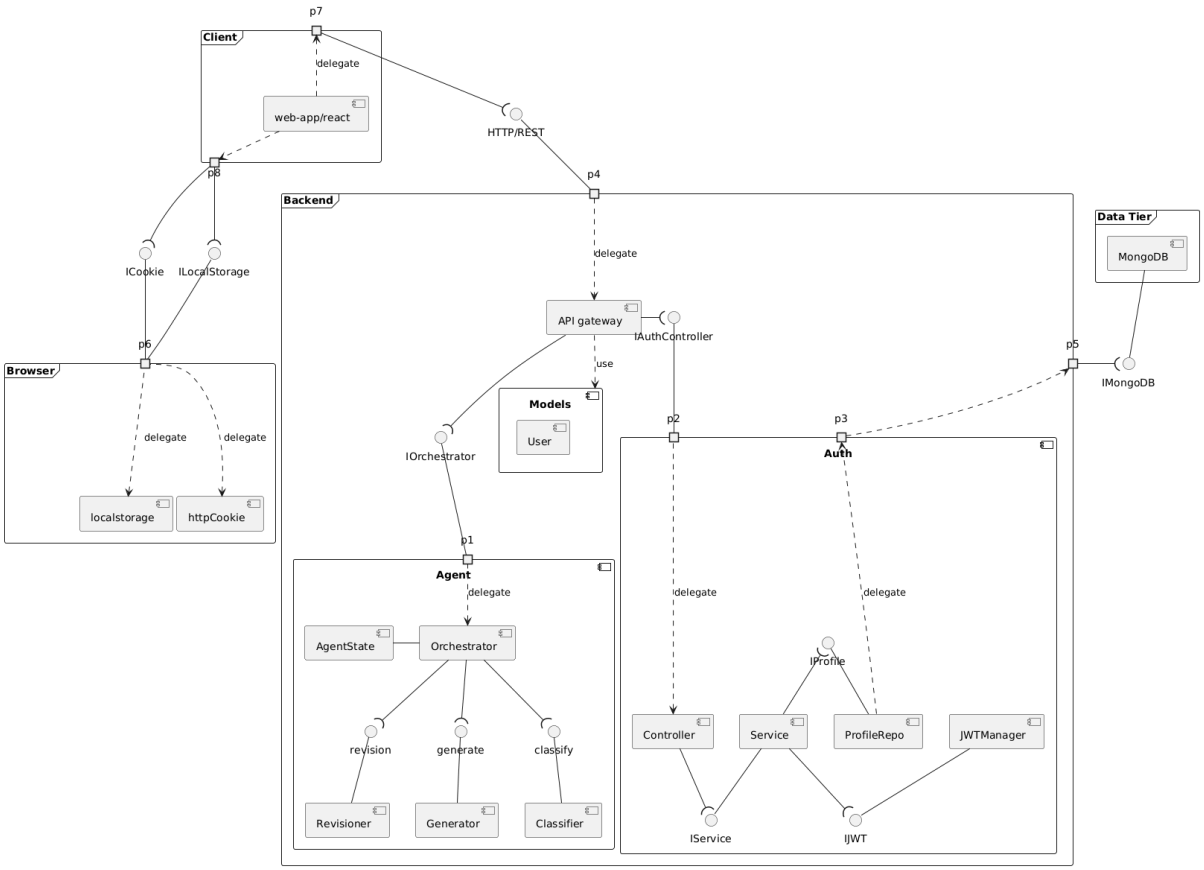

Figura 5.2: Opzione 1 – Componenti con memoria conversazionale su `localStorage`

Il diagramma mostra un frontend responsabile della memoria locale: lo storico rimane nel browser e viene inviato al backend insieme alla richiesta, mantenendo il server stateless.

5.3.2 Opzione 2 – Redis lato server + persistenza su MongoDB

In questa soluzione il backend mantiene la memoria conversazionale attiva in Redis (veloce, in-memory) e salva periodicamente o per evento la conversazione su MongoDB per persistenza a lungo termine.

Vantaggi:

- Ottime performance in lettura/scrittura del contesto conversazionale
- Memoria condivisa lato server, indipendente dal client
- Persistenza storica su MongoDB

Svantaggi:

- Maggiore complessità architetturale e di manutenzione
- Costo infrastrutturale più elevato (Redis + logica di sincronizzazione)
- Necessità di gestione sessioni lato backend

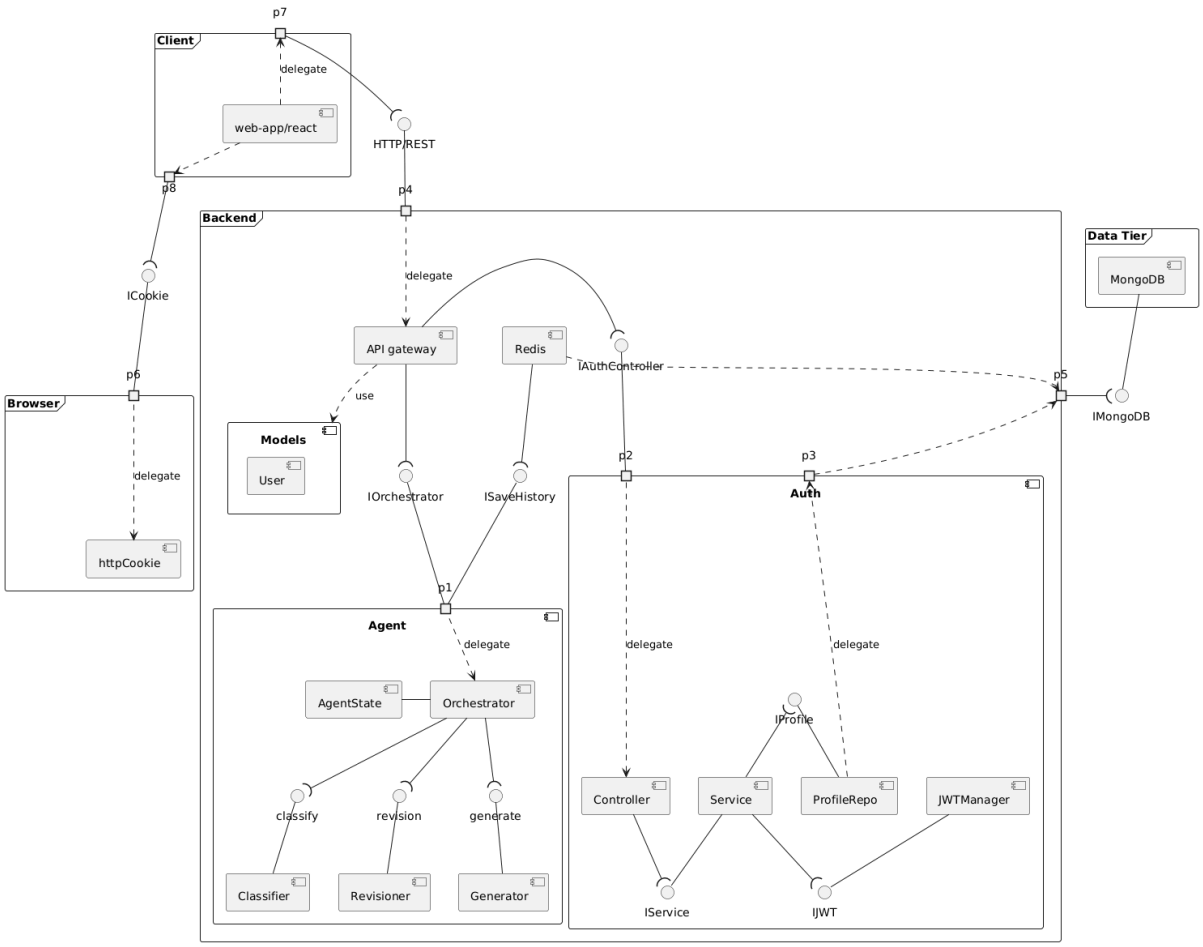

Figura 5.3: Opzione 2 – Componenti con Redis per memoria attiva e MongoDB per persistenza

Il diagramma evidenzia una memoria conversazionale centralizzata lato server: Redis gestisce il contesto rapido della sessione, mentre MongoDB conserva lo storico persistente.

5.3.3 Opzione 3 – Salvataggio turn-by-turn solo su MongoDB

Questa opzione prevede di salvare ogni richiesta/risposta direttamente su MongoDB e ricostruire il contesto a ogni nuova query leggendo lo storico più recente dal database.

Vantaggi:

- Architettura più semplice rispetto a Redis+MongoDB
- Persistenza completa e immediata di ogni turno conversazionale
- Tracciabilità elevata

Svantaggi:

- Aumento latenza
- Necessità di query frequenti per ricostruire il contesto
- Maggiore crescita dei dati nel lungo periodo

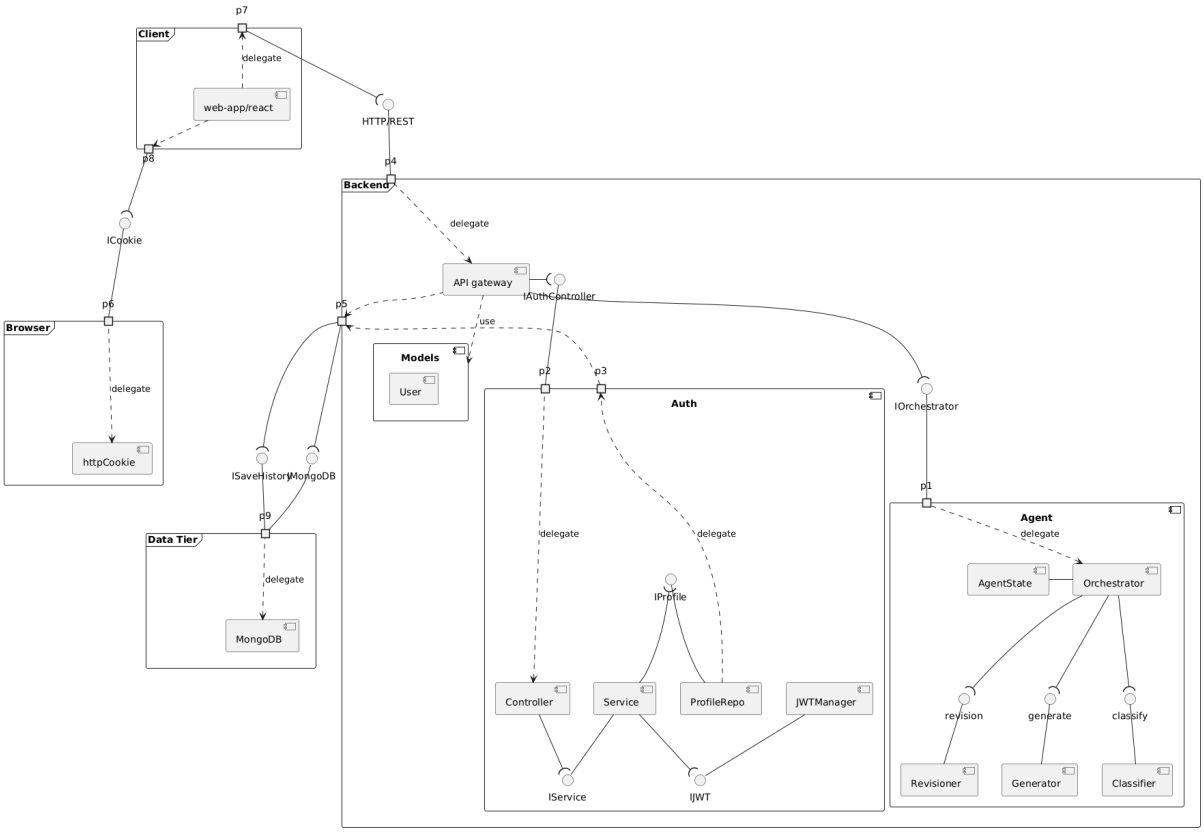

Figura 5.4: Opzione 3 – Componenti con persistenza conversazionale continua su MongoDB

Il diagramma rappresenta un flusso in cui ogni turno viene salvato e riletto da MongoDB, privilegiando la tracciabilità completa rispetto alle prestazioni pure.

5.3.4 Decisione Finale

Per l'attuale fase progettuale abbiamo scelto **l'Opzione 1** (`localStorage`). La scelta è motivata dal miglior compromesso tra semplicità implementativa, tempi di sviluppo e requisiti del progetto in questa iterazione. L'invio della history nel payload garantisce comunque contesto sufficiente agli agenti, mantenendo il backend pulito, stateless e facilmente estendibile verso opzioni server-side nelle iterazioni future.

5.4 Design dell'Architettura

5.4.1 Diagramma delle Classi

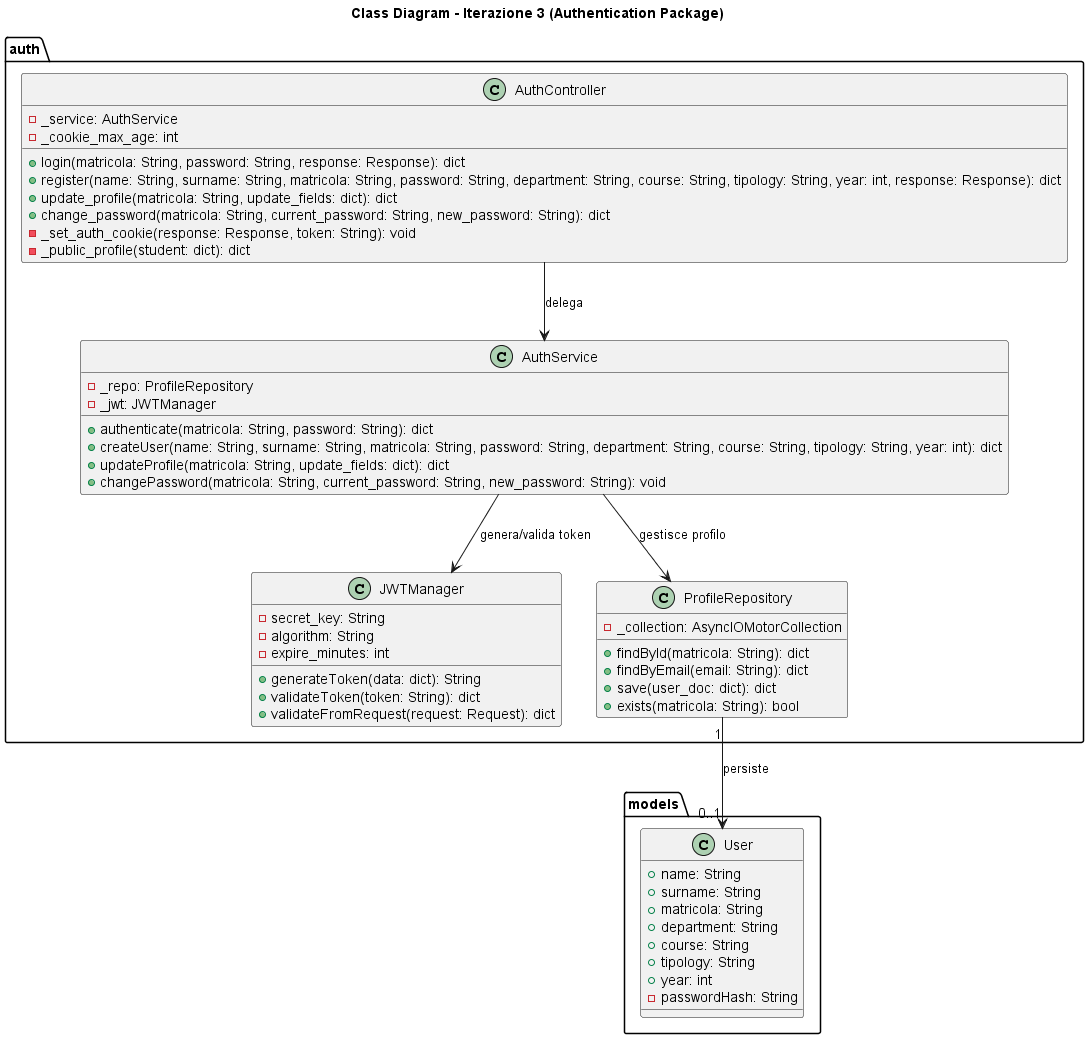

Figura 5.5: Diagramma delle classi – Layer Autenticazione (Iterazione 3)

Il diagramma descrive la separazione dei ruoli nel layer auth (Controller, Service, Repository e JWT Manager), migliorando modularità, testabilità e manutenibilità.

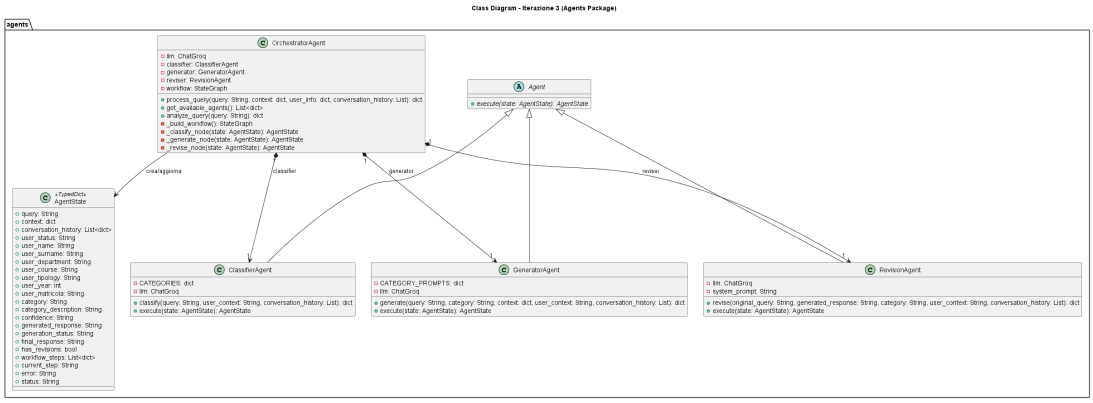

Figura 5.6: Diagramma delle classi – Agenti (Iterazione 3)

Il diagramma mostra le principali classi degli agenti e le loro responsabilità, evidenziando l'orchestrazione della pipeline e l'uso condiviso dello stato conversazionale.

5.4.2 Diagramma di Sequenza

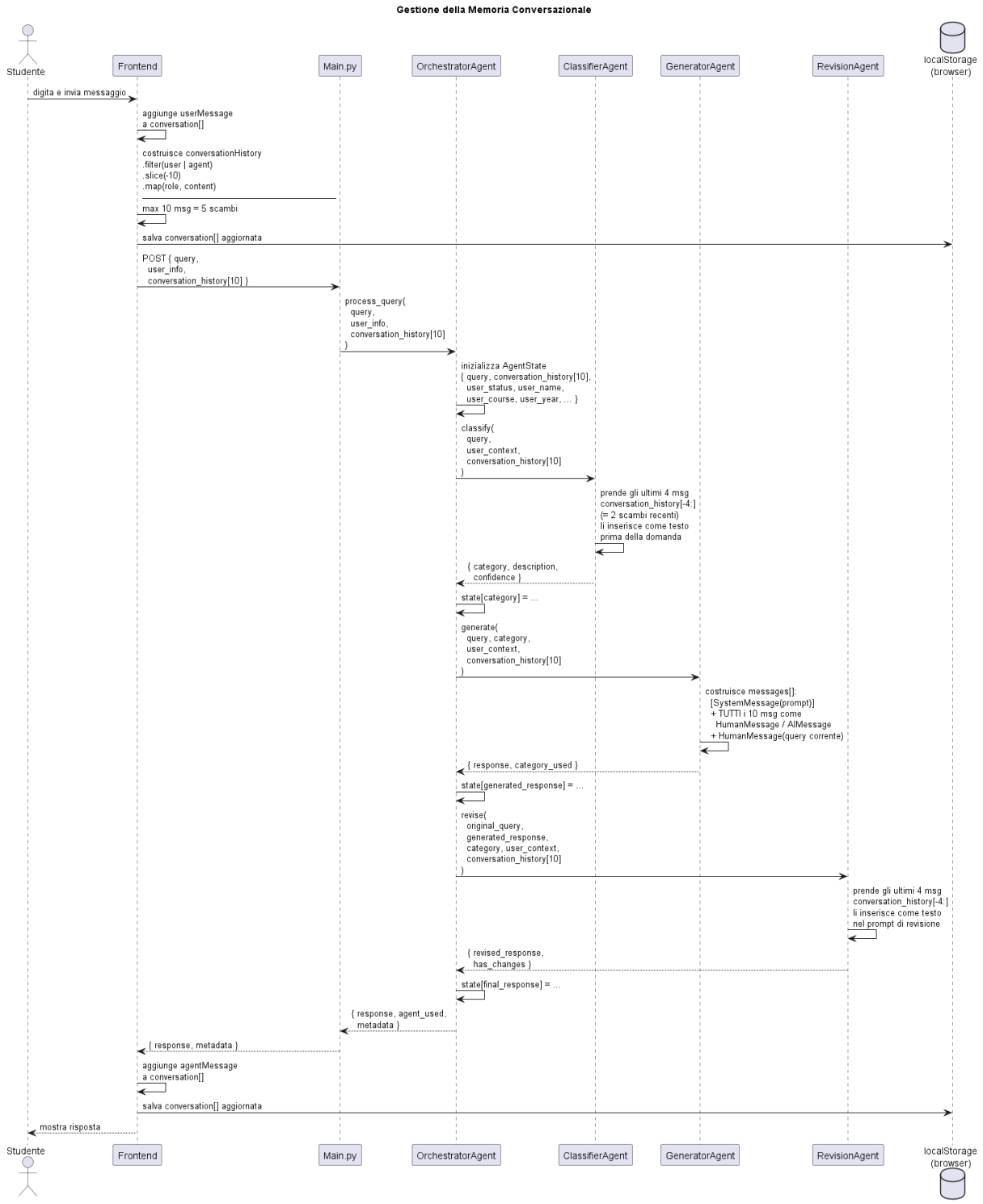

Figura 5.7: Diagramma di sequenza – Query con memoria conversazionale (Iterazione 3)

Il diagramma illustra l'ordine temporale delle interazioni tra frontend, backend e agenti durante una query contestuale, includendo invio history, generazione e revisione risposta.

5.4.3 Diagramma di Attività

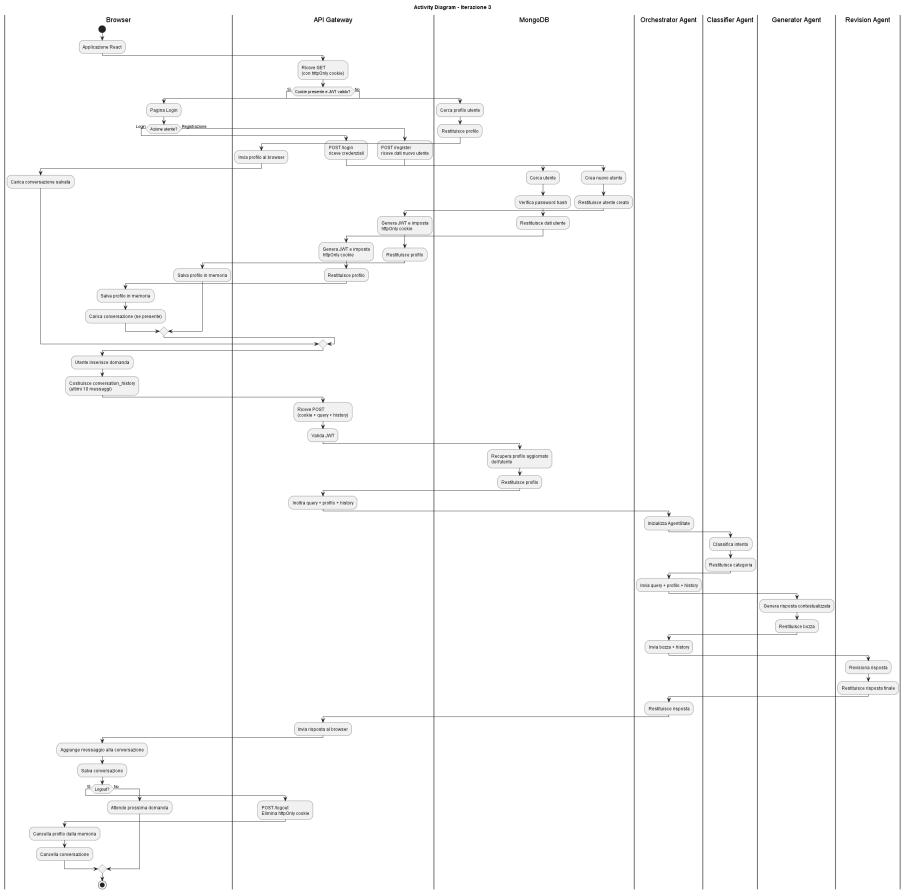

Figura 5.8: Diagramma di attività – Flusso completo con sessione e memoria (Iterazione 3)

Il diagramma sintetizza il flusso decisionale end-to-end: autenticazione, gestione del contesto conversazionale, esecuzione pipeline e restituzione della risposta finale.

5.5 Analisi Dinamica

L'analisi dinamica è condotta tramite test API eseguiti con Postman, estendendo la collection dell'Iterazione 2. Il riferimento a tali test è presente nel file postman-collection.json. Le variabili utilizzate sono sempre le stesse della precedente iterazione.

ID	Endpoint	Met.	Descrizione	Atteso
IT3-01	`/api/agent/query`	POST	Prima domanda senza storico	200 OK
IT3-02	`/api/agent/query`	POST	Follow-up con contesto	200 OK
IT3-03	`/api/agent/query`	POST	Cambio argomento	200 OK
IT3-04	`/api/conversation/history`	DELETE	Cancellazione cronologia	200 OK

Tabella 5.1: Test API – Iterazione 3

Il sistema disambigua correttamente domande di follow-up utilizzando il `conversation_history`. Un cambio di argomento viene gestito correttamente senza confusione con il contesto precedente.

Capitolo 6

Iterazione 4 – Web Search, Query Agent e Logger

6.1 Obiettivo dell'Iterazione

L'obiettivo dell'iterazione è arricchire il sistema con la capacità di recuperare informazioni reali e aggiornate dal sito dell'Università di Bergamo (utilizzando il dominio unibg.it), attraverso l'utilizzo di un prompt adeguato per la ricerca e la creazione di un logger conversazionale per seguire i passaggi effettuati dal sistema.

- **QueryAgent**: agente dedicato alla generazione di query di ricerca web ottimizzate a partire dalla domanda dell'utente, dalla cronologia della conversazione e dal profilo dello studente.
- **WebAgent**: agente che esegue ricerche web tramite Tavily API su domini specifici, estraendo i contenuti testuali dei top 5 risultati e gestisce anche documenti PDF tramite PyMuPDF.
- **PipelineLogger**: componente che scrive un file di log dettagliato per ogni query elaborata, tracciando tutti i passi della pipeline (classificazione, search query, risultati web, generazione, revisione).
- **Cambio LLM**: da Groq / LLaMA 3.3 70B a Google Gemini 2.5 Flash per tutti gli agenti.

6.2 Analisi dei Requisiti e Use Case

6.2.1 Use Cases

- **UC-13 – Risposta con Dati Reali da UniBG**: classify → QueryAgent (6-8 keyword) → WebAgent Tavily search (advanced, max 30) → top 5 + PyMuPDF PDF links → GeneratorAgent con web_context → RevisionAgent → PipelineLogger.
- **UC-14 – Logging Diagnostico**: ogni esecuzione genera un file `log_{matricola}_{timestamp}.t` con: header, user info, query, history, step 1-5 (prompt + risposta).

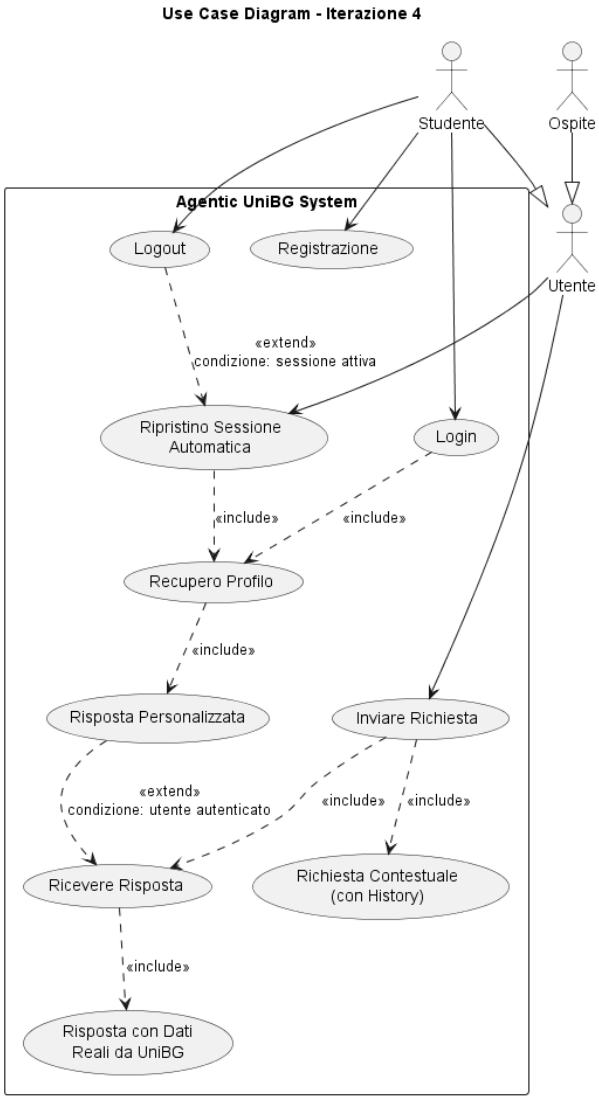

Figura 6.1: Diagramma dei casi d'uso – Iterazione 4

6.2.2 Requisiti Non Funzionali

- **RNF-5**: ricerca limitata a `unibg.it` e `unibg.coursecatalogue.cineca.it` per garantire che i dati provengano da fonti ufficiali dell'ateneo.
- **RNF-6**: fallback su risultati parziali se Tavily fallisce, garantendo comunque una risposta finale.
- **RNF-7**: gestione PDF opzionale, l'estrazione del link non deve essere bloccante se non è possibile.
- **RNF-8**: Il web-context iniettato nel prompt del GeneratorAgent deve essere limitato ai top 5 risultati ottenuti.
- **RNF-9**: Il QueryAgent deve produrre keyword compatte (6-8 termini) per massimizzare la rilevanza dei risultati Tavily e minimizzare i costi API. L'inclusione del profilo studente nelle keyword deve essere una decisione autonoma del modello, in base alla rilevanza.

6.3 Design dell'Architettura

6.3.1 Diagramma dei Componenti

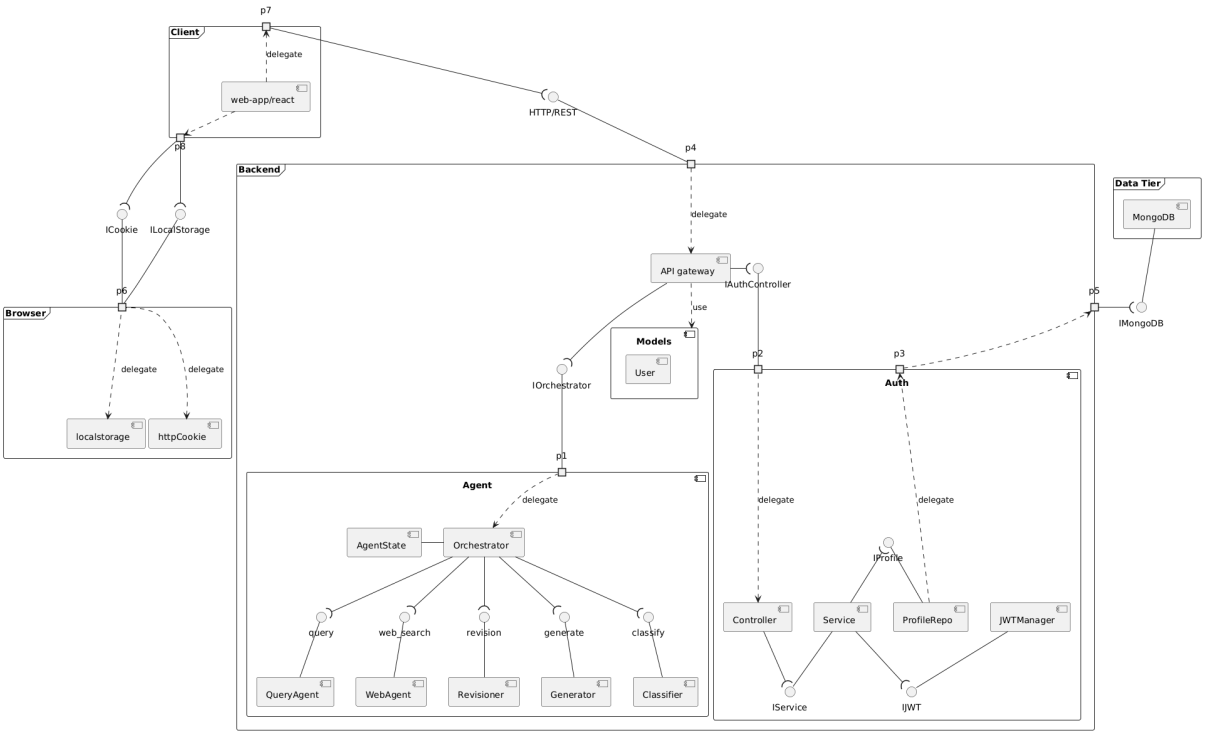

Figura 6.2: Diagramma dei componenti – Architettura completa (Iterazione 4)

Come si può notare dall'immagine, rispetto all'iterazione 3 è stato aggiunto il componente QueryAgent, con l'interfaccia `query`, che riceve la domanda dell'utente dall'Orchestrator e la trasforma in una query di ricerca web ottimizzata, tenendo conto del profilo utente e della conversation history per contestualizzare la ricerca. È stato inoltre introdotto il componente WebAgent, con l'interfaccia `web_search`, che riceve la query ottimizzata dal QueryAgent e interroga l'API di Tavily per recuperare informazioni reali dal sito dell'ateneo, selezionando i 5 risultati migliori ed estraendo link e PDF.

Perchè Tavily API?

Tavily è un motore di ricerca avanzato basato su IA, progettato specificamente per agenti AI e LLM, che fornisce dati in tempo reale, estrazione di contenuti e web scraping. Abbiamo scelto di configurare Tavily con ricerca "advanced", recupero fino a 30 risultati, selezione dei 5 risultati con score più alto ed estrazione del testo e dei collegamenti delle pagine ottenute.

```python
async def search(self, search_query: str) -> Dict:
    """
    Esegue una ricerca web con Tavily sulla query fornita,
    recupera 30 risultati e restituisce i top 5 per score.
    """
    try:
        response = self.client.search(
            query=search_query,
            search_depth="advanced",
            include_domains=self.SEARCH_DOMAINS,
            max_results=30,
            chunks_per_source=5,
            include_raw_content=True
        )

        results = response.get("results", [])
```

Figura 6.3: Ricerca di Tavily (Iterazione 4)

Perchè Google Gemini?

Il provider LLM è stato cambiato da Groq (llama-3.3-70b) a Google Gemini 2.5 flash perchè:

Aspetto	Groq / LLaMA 3.3 70B	Google Gemini 2.5 Flash
Latenza	Molto basse	Bassa
Context window	128K token	1M token
Qualità ragionamento	Alta	Molto alta
Gestione contesti lunghi	Limitata	Ottima (utile con web context)

Tabella 6.1: Confronto LLM provider

6.3.2 Diagramma delle Classi

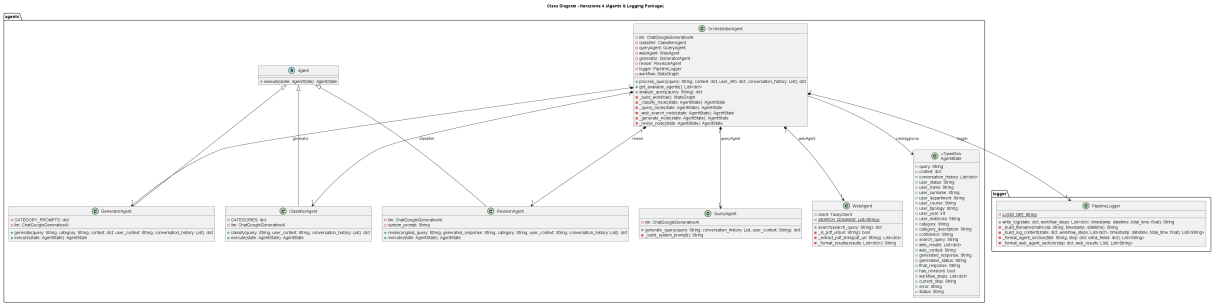

Figura 6.4: Diagramma delle classi – Agenti (Iterazione 4)

Il diagramma delle classi dell'iterazione 4, mostra l'aggiunta delle classi QueryAgent e WebAgent, con i rispettivi metodi, che interagiscono nella pipeline con l'OrchestratorAgent.

Query Agent

Il Query Agent trasforma la domanda dell'utente in una query di ricerca web breve e ottimizzata. L'agente decide autonomamente se includere le informazioni del profilo studente (corso, anno e tipologia) nella query. Il contesto della conversazione ha sempre la priorità sul profilo dello studente. Se la conversazione indica che l'utente sta chiedendo informazioni su un corso/dipartimento diverso dal proprio, la query deve riferirsi ai dati menzionati nella conversazione e non a quelli del profilo.

Web Agent

Il Web Agent esegue ricerche su Tavily API limitando i risultati ai domini universitari e prepara il contenuto per il Generator. Se un risultato punta a un URL .pfd, l'agente scarica in memoria il pdf, apre il documento con PyMuPDF (libreria Python), estrae tutti i link ipertestuali da ogni pagine e i link vengono inclusi nel contesto formattato passato al Generator.

Pipeline Logger

Scrive un file .txt nella cartella logs per ogni query elaborata, con tracciamento completto di tutti i passi della pipeline. Ogni file contiene le seguenti sezioni:

1. Header: timestamp, status pipeline.
2. User Information: tipo utente, nome, corso, anno, dipartimento.
3. Original User Query.
4. Conversation History: ultimi 3 turni.
5. Classifier Agent: categoria, descrizione, confidence.
6. Query Agent: search query generata, query originale.
7. Web Agent: risultati Tavily.
8. Generator Agent: risposta generata.

9. Revision Agent: risposta finale, modifiche apportate.

6.3.3 Diagramma di Sequenza

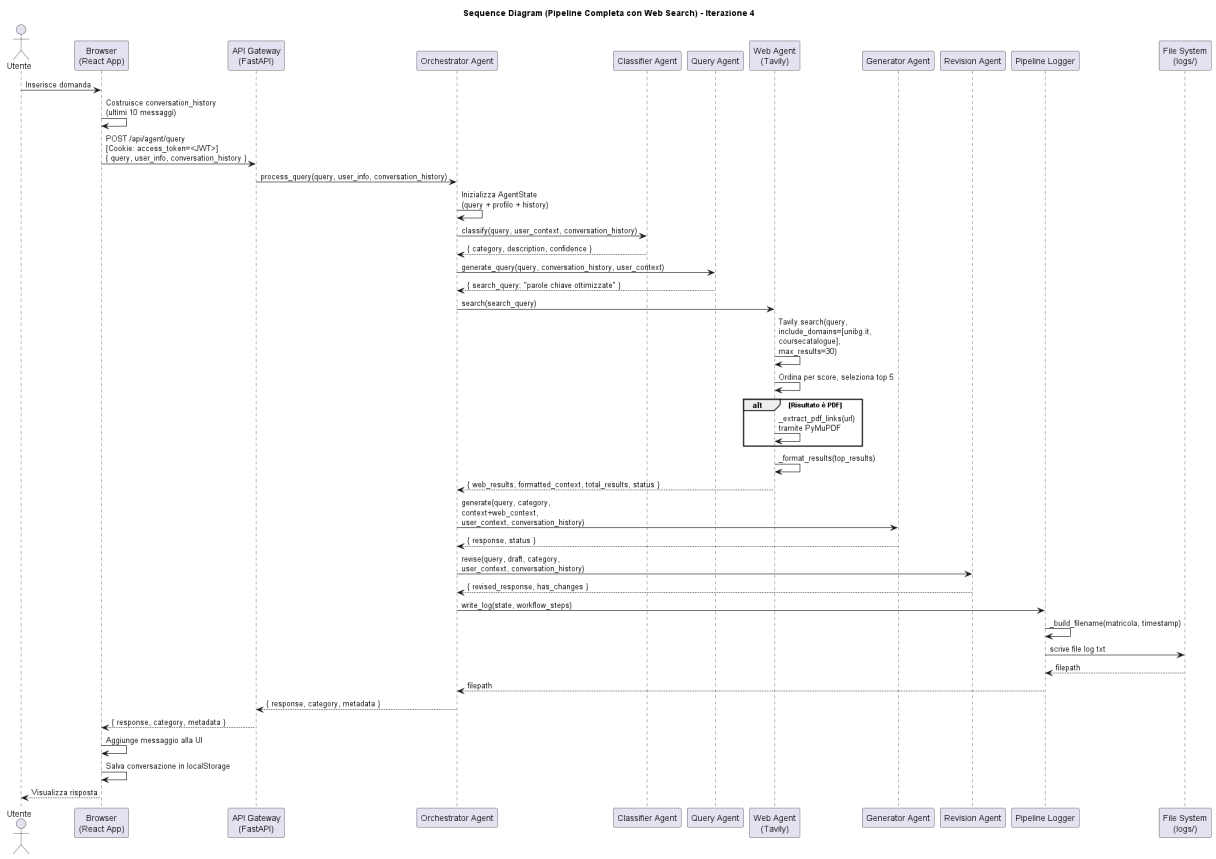

Figura 6.5: Diagramma di sequenza – Pipeline completa con web search (Iterazione 4)

Il diagramma di sequenza mostra la pipeline completa dall'inserimento di una domanda, passando per la ricerca web, fino alla ricezione della risposta e alla creazione dei file di log.

6.3.4 Diagramma di Attività

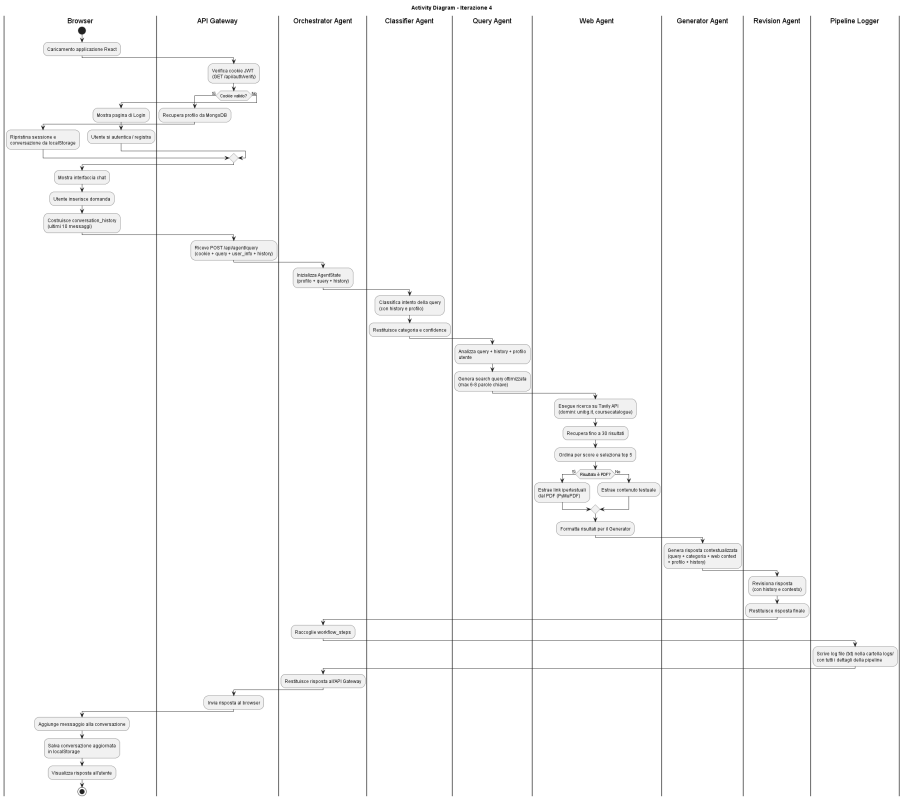

Figura 6.6: Diagramma di attività – (Iterazione 4)

L'activity diagram dell'Iterazione 4 mostra il flusso completo di una richiesta utente attraverso il sistema multi-agente. Il flusso è il seguente: Browser → API Gateway (autenticazione JWT) → Orchestrator → Classifier (classifica intento) → Query Agent (ricerca documenti universitari via API, filtra top 5, estrae da PDF) → Generator (genera risposta contestualizzata) → Revision (revisione qualità) → Pipeline Logger (logging) → risposta restituita al browser.

6.4 Analisi Dinamica

Per validare il comportamento dell'Iterazione 4 sono stati eseguiti test end-to-end sugli endpoint principali della pipeline conversazionale. L'obiettivo è verificare non solo la correttezza della risposta HTTP, ma anche la coerenza del flusso interno tra QueryAgent e WebAgent, con particolare attenzione alla qualità della query ottimizzata e alla stabilità della web search.

ID	Endpoint	Met.	Descrizione	Atteso
IT4-01	`/api/agent/query`	POST	Query con web search	200 OK
IT4-02	`/api/agent/query`	POST	Query Agent genera query ottimizzata	200 OK

Tabella 6.2: Test API – Iterazione 4

I risultati ottenuti confermano che la pipeline risponde correttamente nei casi coperti, mantenendo tempi accettabili e output consistenti con le aspettative funzionali.

Capitolo 7

Iterazione 5 – Gestione Profilo, Timestamp Temporale, Date degli Esami e Prompt Engineering

7.1 Obiettivo dell'Iterazione

Consolidare e raffinare la pipeline multi-agente con interventi trasversali su qualità delle risposte, consapevolezza temporale, gestione del profilo utente, introduzione del flusso dedicato per le date degli esami e miglioramento del logging:

- **Pagina Impostazioni Profilo Utente**: nuovo componente React `SettingsPage` per la modifica dei dati personali e il cambio password, con due nuovi endpoint REST (`PUT /api/auth/profile` e `PUT /api/auth/password`).
- **Timestamp Temporale per gli Agenti**: tutti gli agenti ricevono la data corrente in formato italiano tramite `get_italian_timestamp()`, consentendo risposte temporalmente consapevoli (anno accademico, prossima sessione, scadenze).
- **Routing Condizionale** `date_esami`: il `ClassifierAgent` riconosce la nuova categoria `date_esami` per domande relative a date, sessioni e calendari degli esami. Il workflow LangGraph introduce un **branch condizionale** dopo il nodo `query`: se la categoria è `date_esami`, il flusso viene dirottato verso un nodo dedicato `exam_extract` (anziché `web_search`), dove il `WebAgent` seleziona autonomamente tramite LLM il calendario corretto dalla struttura statica `EXAM_CALENDAR_LINKS` e ne estrae il contenuto integrale con `Tavily Extract`. Il prompt di selezione gestisce esplicitamente il caso ospite senza corso specificato (`NUMERO: 0`).
- **Prompt Engineering Estensivo**: revisione sistematica di tutti i prompt di sistema (ClassifierAgent, QueryAgent, GeneratorAgent, RevisionAgent) con test manuali iterativi e correzioni mirate. Include `SOURCE_INSTRUCTIONS` per l'uso delle fonti web, history come messaggi LLM nativi e regole di formato.
- **Tempi di Esecuzione nel Logger**: il `PipelineLogger` registra ora il tempo di ogni singolo agente e il tempo totale della pipeline, con sezione dedicata nel file di log tramite `_format_timing_section()`.

7.2 Analisi dei Requisiti e Use Case

7.2.1 User Stories

ID	Descrizione	Ruolo
US-10	Come studente, voglio che il sistema conosca la data odierna così da ricevere risposte temporalmente accurate (es. "prossima sessione", "scadenza iscrizione")	Studente
US-11	Come studente, voglio poter modificare i miei dati personali (nome, cognome, dipartimento, corso, anno) dalla pagina impostazioni senza dovermi re-iscrivere	Studente
US-12	Come studente, voglio poter cambiare la mia password dall'interfaccia web in modo sicuro	Studente
US-13	Come studente, voglio risposte più precise e meglio strutturate grazie a prompt ottimizzati che gestiscano correttamente il contesto della conversazione	Studente
US-14	Come sviluppatore, voglio monitorare i tempi di esecuzione di ogni agente nei log per identificare colli di bottiglia nella pipeline	Sistema
US-15	Come studente, voglio chiedere le date degli esami e ricevere informazioni estratte direttamente dal calendario ufficiale del mio polo	Studente

Tabella 7.1: User stories – Iterazione 5

7.2.2 Use Cases

- **UC-15 – Modifica Profilo Utente**

  Descrizione: L'utente clicca il pulsante ingranaggio nell'header della chat. Si apre il pannello `SettingsPage` con il form "Dati Profilo" precompilato con i dati correnti. L'utente modifica uno o più campi (nome, cognome, dipartimento, corso, anno). Quando l'utente cambia dipartimento, il dropdown del corso mostra solo i corsi di quel dipartimento (da `DIPARTIMENTI_CORSI`); la tipologia viene impostata automaticamente in base al tipo del corso selezionato. L'utente clicca "Salva Modifiche". Il frontend invia `PUT /api/auth/profile` con i campi modificati e il cookie JWT. Il backend estrae la matricola dal token, delega ad `AuthService.updateProfile()` che verifica l'esistenza dell'utente e chiama `ProfileRepository.updateProfile()`, il quale esegue `update_one` con `$set` dei campi forniti su MongoDB. Il profilo aggiornato (senza `passwordHash`) viene restituito al frontend, che aggiorna `userInfo` nello stato React tramite la callback `onProfileUpdate`.

- **UC-16 – Cambio Password**

  Descrizione: L'utente compila il form "Cambia Password" nella `SettingsPage`: password corrente, nuova password (minimo 6 caratteri) e conferma. Il frontend valida localmente che tutti i campi siano compilati, che la nuova password abbia almeno 6 caratteri e che la conferma corrisponda. Il frontend invia `PUT /api/auth/password` con `current_password` e `new_password` e il cookie JWT.

Il backend estrae la matricola dal token, delega ad `AuthService.changePassword()`, che recupera il profilo da MongoDB e verifica la password corrente con `bcrypt.checkpw()`. Se corretta, la nuova password viene hashata con `bcrypt.hashpw()` e aggiornata su MongoDB. Il backend restituisce `{status: "success"}` e il frontend mostra il messaggio di successo.

- **UC-17 – Risposta Temporalmente Consapevole**

  Descrizione: La pipeline viene attivata dall'`OrchestratorAgent`. Al nodo `query`, il `QueryAgent` invoca `get_italian_timestamp()` che restituisce la data corrente in formato italiano (es. "06 marzo 2026"). Il system prompt include `DATA ODIERNA: {today} (Anno accademico 2025/2026)`, consentendo all'agente di includere riferimenti temporali nella search query. Al nodo `generate`, il `GeneratorAgent` invoca nuovamente `get_italian_timestamp()`: può così determinare "prossima sessione", "scadenza imminente" e "periodo corrente". Se attivato, il nodo `exam_extract` include la data odierna nel contesto del calendario esami tramite `_format_exam_results()`.

- **UC-18 – Risposta su Date Esami con Calendario Ufficiale**

  Descrizione: Il routing condizionale del grafo LangGraph instrada la richiesta verso il nodo `exam_extract` anziché `web_search`. Il `WebAgent` recupera tutti i calendari esami disponibili dalla struttura `EXAM_CALENDAR_LINKS` (link statici organizzati per polo/dipartimento e sessione). L'agente costruisce un prompt dedicato contenente la domanda dell'utente, il suo profilo (se autenticato), la data odierna e la lista numerata dei calendari disponibili (con opzione "0" per il caso indeterminabile). Il prompt viene inviato all'LLM, che seleziona il calendario più appropriato. Se l'utente è un ospite e non ha specificato un corso o dipartimento nella domanda, l'LLM risponde con NUMERO: 0 (nessun calendario selezionato); il `GeneratorAgent` chiederà gentilmente all'utente di specificare il proprio corso. Se un calendario viene selezionato, il `WebAgent` usa `Tavily.extract()` per estrarre il contenuto integrale della pagina/PDF del calendario. Parallelamente, viene eseguita una ricerca Tavily standard per ottenere 3 fonti web aggiuntive. Il contesto formattato (`_format_exam_results()`) viene passato al `GeneratorAgent`, che cerca le date specifiche richieste dall'utente nel calendario e produce la risposta.

- **UC-19 – Logging con Tempi di Esecuzione**

  Descrizione: Ogni nodo del workflow LangGraph misura il proprio tempo di esecuzione tramite `time.time()` e lo registra nel campo `elapsed_time` del rispettivo `workflow_step`. Il tempo totale della pipeline viene calcolato nell'`OrchestratorAgent.proces`
  Il `PipelineLogger` scrive nel file di log una sezione "TEMPI DI ESECUZIONE" con il tempo di ogni agente (3 cifre decimali) e il tempo totale.

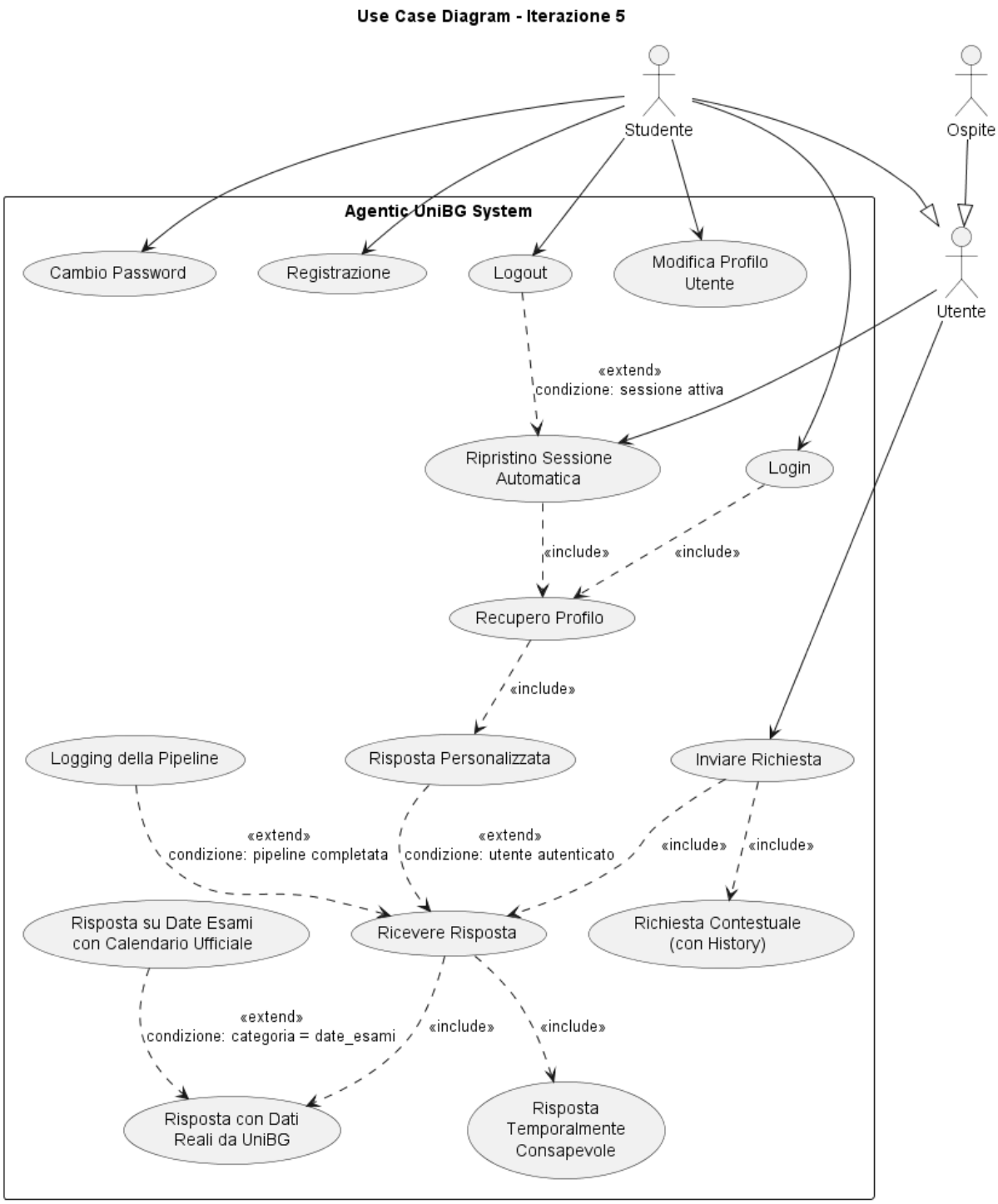

Figura 7.1: Diagramma dei casi d'uso – Iterazione 5

Il diagramma evidenzia i tre attori: lo **Studente** (autenticato, con accesso a Modifica Profilo Utente, Cambio Password, Risposta Personalizzata e Logging della Pipeline), l'**Ospite** (accesso diretto alla chat senza profilazione) e l'attore generalizzato **Utente**. Rispetto all'Iterazione 4, vengono aggiunti i casi d'uso "Modifica Profilo Utente" e "Cambio Password" (riservati allo Studente). "Risposta Temporalmente Consapevole" include "Ricevere Risposta". Il caso d'uso "Risposta su Date Esami con Calendario Ufficiale" estende "Ricevere Risposta" con la condizione `categoria = date_esami`, e a sua volta

"Risposta con Dati Reali da UniBG" estende "Risposta su Date Esami" per il recupero dei dati dal calendario ufficiale.

7.2.3 Requisiti Non Funzionali

- **RNF-10 – Validazione lato client**: la `SettingsPage` valida che dipartimento e corso siano selezionati dal dizionario `DIPARTIMENTI_CORSI`; la tipologia è derivata automaticamente e non editabile; l'anno massimo dipende dalla tipologia (3, 2 o 5).
- **RNF-11 – Sicurezza cambio password**: verifica della password corrente con `bcrypt.checkpw()` obbligatoria; nuova password hashata con `bcrypt.hashpw()` e `bcrypt.gensalt()`; lunghezza minima 6 caratteri (validata client e server).
- **RNF-12 – Precisione temporale**: `get_italian_timestamp()` restituisce la data in formato "dd mese yyyy" con nome del mese in italiano; precisione al giorno.
- **RNF-13 – Priorità contesto conversazionale**: `QueryAgent` e `GeneratorAgent` danno sempre priorità al contesto della conversazione rispetto al profilo studente.
- **RNF-14 – Gestione fonti web**: quando presente `web_context`, il `GeneratorAgent` segue le `SOURCE_INSTRUCTIONS` (8 regole per l'uso delle fonti); il `RevisionAgent` preserva gli URL in fondo alla risposta.

7.3 Design dell'Architettura

7.3.1 Diagramma dei Componenti

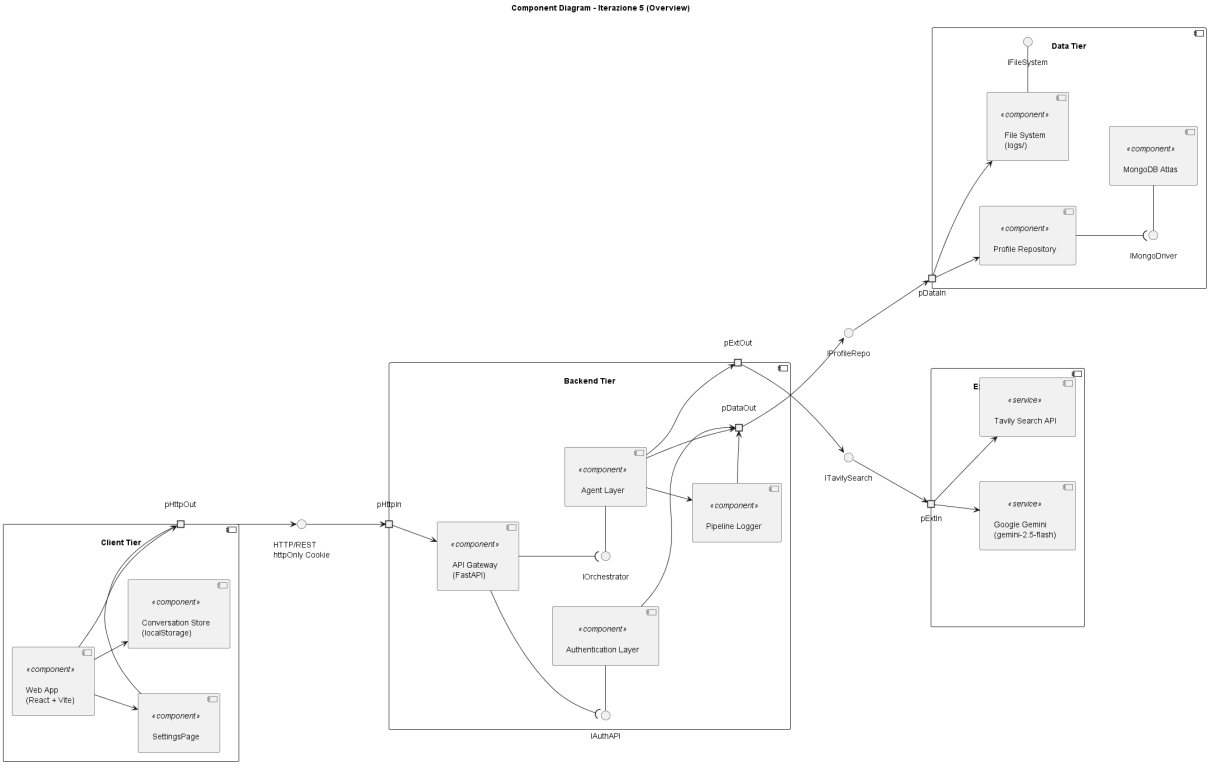

Figura 7.2: Diagramma dei componenti – Architettura completa, overview (Iterazione 5)

L'architettura resta articolata in tre tier. Rispetto all'Iterazione 4, il Client Tier include il nuovo componente `SettingsPage` e il Backend Tier espone due nuovi endpoint nell'Authentication Layer (`PUT /api/auth/profile` e `PUT /api/auth/password`). L'Agent Layer introduce il routing condizionale per la categoria `date_esami`.

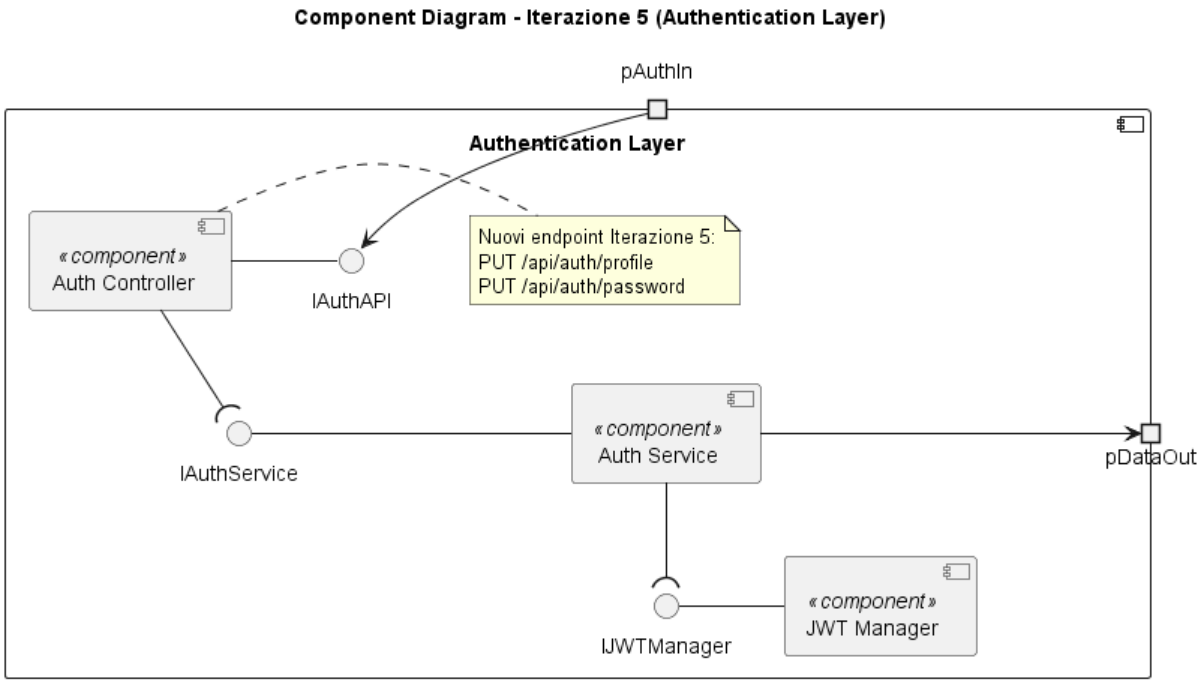

Figura 7.3: Diagramma dei componenti – Authentication Layer (Iterazione 5)

Il diagramma dell'Authentication Layer mostra i due nuovi endpoint esposti dall'`Auth Controller` tramite l'interfaccia `IAuthAPI`: `PUT /api/auth/profile` e `PUT /api/auth/password`. Il flusso attraversa `AuthService` e raggiunge `ProfileRepository` per la persistenza su MongoDB, con `JWTManager` per la verifica del token.

Workflow LangGraph

Il workflow include un **branch condizionale** dopo il nodo `query`, introdotto in questa iterazione:

```
classify → query → [routing condizionale] → generate → revise → END
                    |-- web_search      (tutte le categorie)
                    `-- exam_extract    (solo date_esami)
```

Il metodo `_route_after_query(state)` verifica `state["category"] == "date_esami"` e instrada il flusso verso `exam_extract` oppure verso `web_search`. Entrambi i rami convergono su `generate`.

7.3.2 Diagramma delle Classi

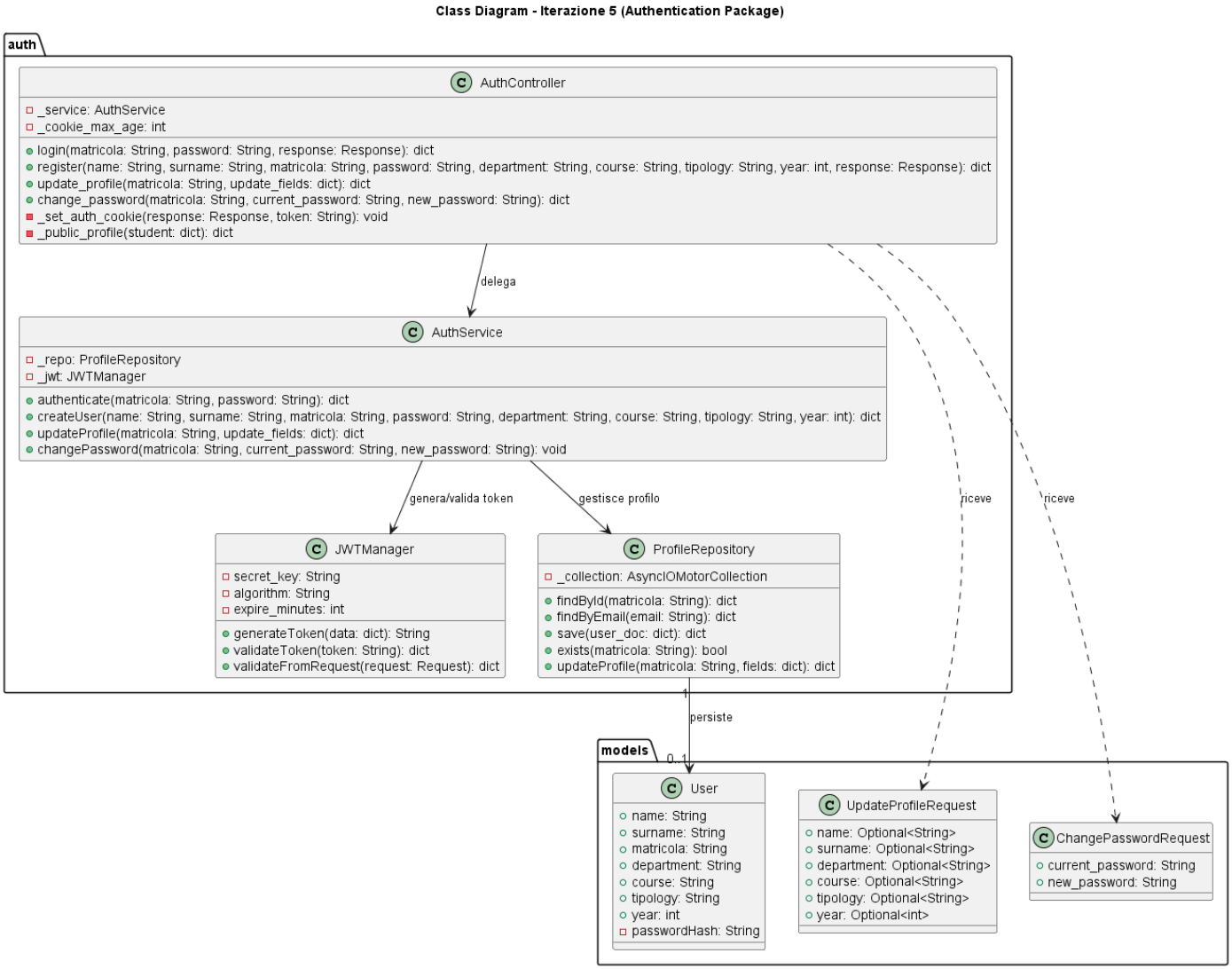

Figura 7.4: Diagramma delle classi – Authentication Package (Iterazione 5)

Rispetto all'Iterazione 4, il layer di autenticazione si arricchisce di:

- `AuthController`: nuovi metodi `update_profile(matricola, update_fields)` e `change_password(matricola, current_password, new_password)`.
- `AuthService`: nuovi metodi `updateProfile(matricola, update_fields)` e `changePassword(ma` `current_password, new_password)`.
- `ProfileRepository`: nuovo metodo `updateProfile(matricola, fields)` che esegue `update_one` con `$set` su MongoDB.

Il pattern Controller → Service → Repository viene esteso senza modifiche strutturali.

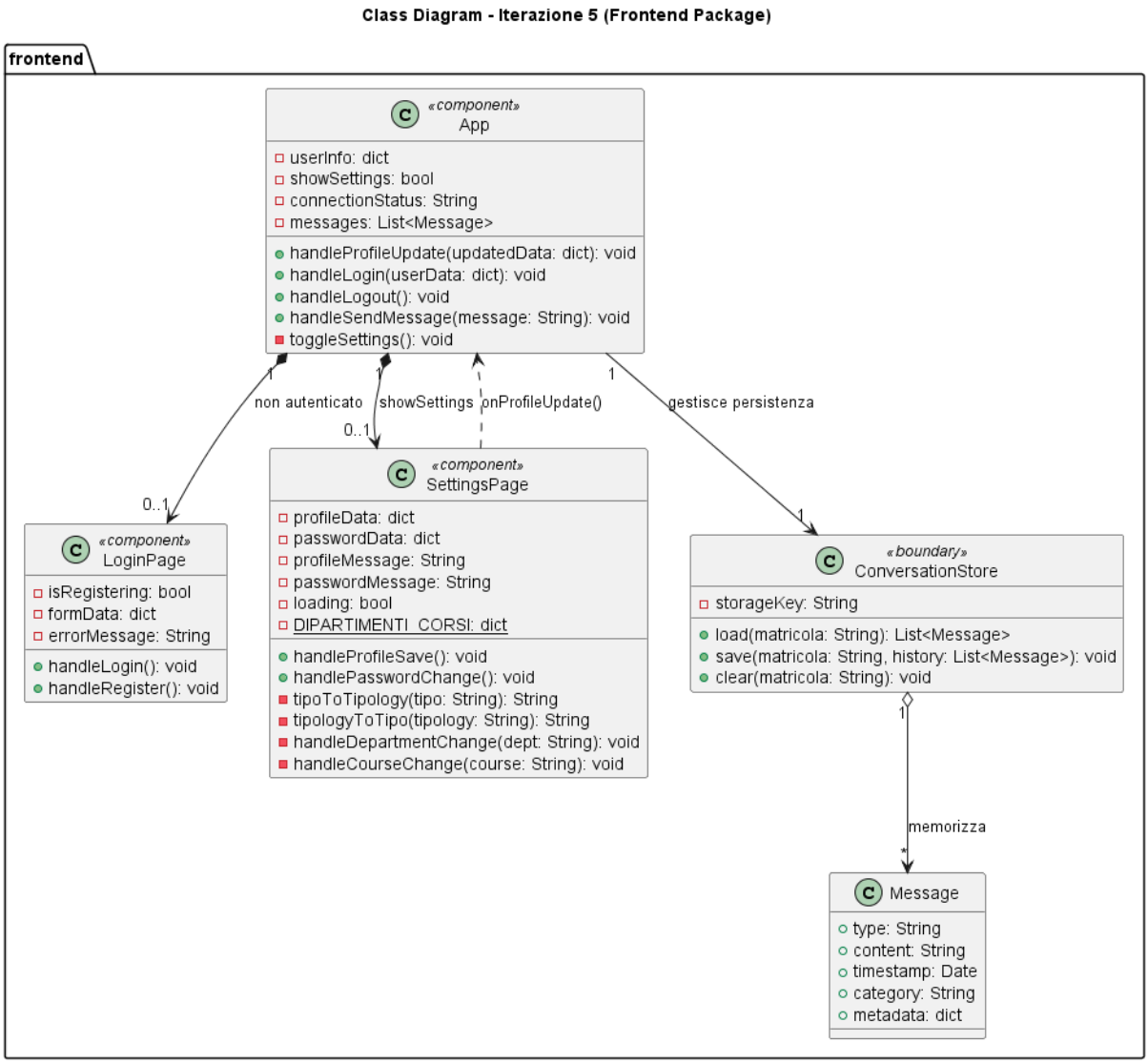

Figura 7.5: Diagramma delle classi – Frontend Package (Iterazione 5)

Il diagramma del frontend introduce il componente `SettingsPage`, collegato ad `App` tramite le relazioni "showSettings" e "onProfileUpdate". `SettingsPage` gestisce lo stato locale (`profileData`, `passwordData`, messaggi di feedback, `loading`) e utilizza il dizionario `DIPARTIMENTI_CORSI` per la selezione gerarchica dipartimento → corso → tipologia. Il componente `ConversationStore` (boundary) gestisce la persistenza della conversazione su `localStorage` come nelle iterazioni precedenti.

7.3.3 Diagramma di Sequenza

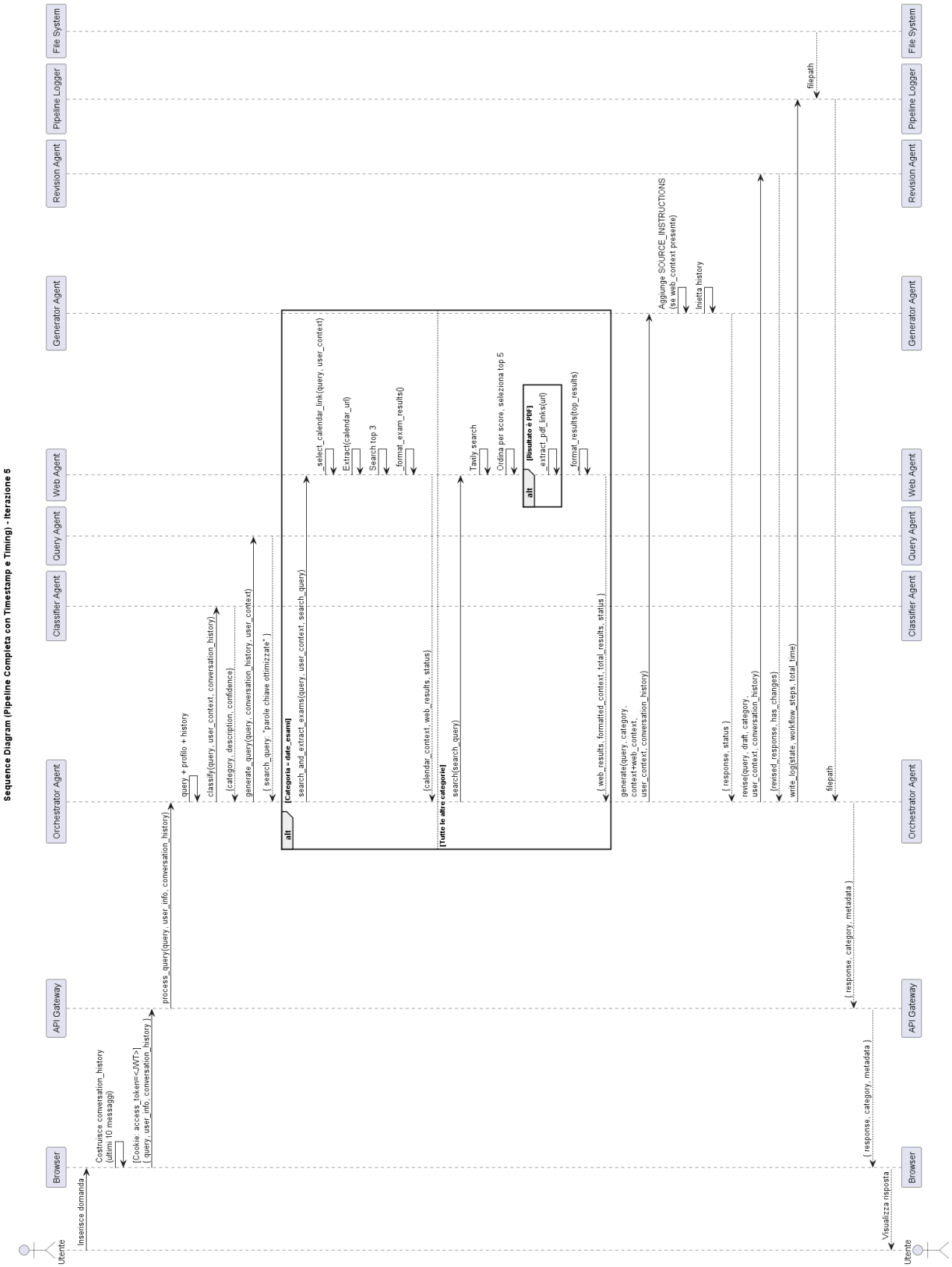

Figura 7.6: Diagramma di sequenza – Pipeline completa con timestamp e timing (Iterazione 5)

Il diagramma di sequenza illustra il flusso completo di una query con le novità dell'Iterazione 5:

1. L'utente invia la query al frontend, che la inoltra all'API Gateway con il cookie JWT e la `conversation_history`.
2. L'API Gateway verifica il token JWT, recupera il profilo da MongoDB e invoca l'`OrchestratorAgent`.
3. L'`OrchestratorAgent` inizializza lo stato e avvia il timer della pipeline.
4. Il `ClassifierAgent` classifica la query usando il contesto conversazionale (ultimi 2 turni) e registra il tempo. Se la categoria è `date_esami`, viene segnalato per il routing condizionale.
5. Il `QueryAgent` genera la search query con `get_italian_timestamp()` e registra il tempo.
6. Il routing condizionale instrada verso `web_search` (categorie standard) o `exam_extract` (categoria `date_esami`).
7. Nel caso `exam_extract`: il `WebAgent` recupera i calendari da `EXAM_CALENDAR_LINKS`, costruisce il prompt con le opzioni numerate e lo invia all'LLM, che seleziona il calendario corretto (o NUMERO: 0 per ospiti senza corso). Se selezionato, `Tavily.extract()` estrae il contenuto del calendario. Parallelamente, una ricerca Tavily fornisce 3 fonti web aggiuntive. Il tempo viene registrato.
8. Il `GeneratorAgent` produce la risposta con timestamp, `SOURCE_INSTRUCTIONS` e, nel caso `date_esami`, il contenuto del calendario estratto. Registra il tempo.
9. Il `RevisionAgent` revisiona applicando le regole di formato (no Markdown, preservazione URL e date) e registra il tempo.
10. Il `PipelineLogger` scrive il file di log con tutti i tempi di esecuzione.
11. La risposta finale viene restituita al frontend.

7.4 Scelte di Design

7.4.1 Routing Condizionale `date_esami` con Selezione LLM del Calendario

L'Iterazione 5 introduce un flusso dedicato per le domande sulle date degli esami, basato su tre componenti: una nuova categoria nel `ClassifierAgent`, un routing condizionale nel grafo LangGraph e una selezione intelligente del calendario tramite LLM.

Il `ClassifierAgent` riconosce la nuova categoria `date_esami` (portando il totale a 7 categorie) per domande relative a date, sessioni, appelli e calendari degli esami. Dopo la classificazione, il metodo `_route_after_query(state)` implementa un branch condizionale: se la categoria è `date_esami`, il flusso salta il nodo `web_search` e viene dirottato verso un nodo dedicato `exam_extract`.

I link ai calendari esami sono organizzati nella costante `EXAM_CALENDAR_LINKS`, strutturata per polo/dipartimento (Scuola di Ingegneria, Polo Economico-Giuridico, Polo Linguistico,

Polo Umanistico) e per sessione (invernale, intermedia/aprile, estiva, autunnale). Il metodo `_select_calendar_link()` costruisce un prompt che presenta all'LLM la lista numerata dei calendari disponibili, insieme alla domanda dell'utente, al suo profilo e alla data corrente. L'LLM sceglie il calendario più appropriato rispondendo con il formato `NUMERO: <n>` e `MOTIVO: <spiegazione>`.

Il prompt include l'opzione "0" (non è possibile determinare il calendario) e una regola critica che impone all'LLM di scegliere NUMERO: 0 quando l'utente è un ospite non autenticato e la domanda non specifica un corso, dipartimento o polo. Questo evita che il sistema scelga un calendario a caso per un utente di cui non si conosce il percorso di studi. Il `GeneratorAgent` riceve il contesto formattato con l'indicazione che il calendario non è stato determinato e chiede gentilmente all'utente di specificare il proprio corso.

Inoltre, il prompt include la regola: "scegli in base alla domanda dell'utente, NON necessariamente in base al profilo". Se uno studente di Ingegneria chiede degli esami di Giurisprudenza, il sistema seleziona il calendario del Polo Economico-Giuridico.

7.4.2 SettingsPage con Dizionario `DIPARTIMENTI_CORSI`

Il dizionario `DIPARTIMENTI_CORSI` è definito nel frontend e mappa ogni dipartimento dell'Università di Bergamo ai rispettivi corsi di laurea con indicazione della tipologia. La selezione gerarchica (dipartimento → corso → tipologia automatica → anno) garantisce coerenza dei dati senza necessità di validazione server-side complessa. La tipologia viene derivata automaticamente tramite `tipoToTipology()`: "Laurea" → "Triennale", "Laurea Magistrale" → "Magistrale", "Laurea Magistrale a ciclo unico" → "Ciclo Unico". L'anno massimo selezionabile dipende dalla tipologia (3, 2 o 5).

7.4.3 Timestamp tramite `get_italian_timestamp()`

La funzione restituisce la data corrente del server in formato italiano (es. "06 marzo 2026") e viene invocata da `QueryAgent`, `GeneratorAgent` e `WebAgent` (`_format_exam_results`). Senza questa informazione, il modello LLM non può rispondere correttamente a domande come "quando è la prossima sessione?" o "la scadenza è già passata?". La scelta di una funzione pura (senza dipendenze esterne) garantisce che non possa mai fallire.

7.4.4 History come Messaggi LLM Nativi

Nell'Iterazione 5, la `conversation_history` non viene più iniettata come stringa testuale nel prompt del `GeneratorAgent`, ma come sequenza di `HumanMessage`/`AIMessage` di LangChain. Questo sfrutta la memoria nativa del modello per risposte più coerenti nei follow-up, rispetto alla serializzazione testuale delle iterazioni precedenti.

7.4.5 Prompt Engineering Iterativo

Il processo di prompt engineering ha seguito un ciclo iterativo: esecuzione di test con domande reali → analisi delle risposte tramite i log della pipeline → identificazione dei pattern di errore (allucinazioni, contesto ignorato, formattazione errata) → modifica mirata del prompt → nuovo ciclo. Le modifiche principali includono:

- **ClassifierAgent**: contesto conversazionale (ultimi 2 turni) per classificazione accurata nei follow-up; 4 esempi specifici per la categoria `date_esami`.
- **QueryAgent**: regola critica "contesto > profilo" e gestione date/sessioni con anno accademico; il contesto della conversazione ha sempre la priorità sulle informazioni del profilo studente.
- **GeneratorAgent**: `SOURCE_INSTRUCTIONS` (8 regole) per l'uso delle fonti web, prompt specializzato `date_esami` con istruzioni per cercare date nel calendario fornito e gestire il caso ospite, timestamp nel prompt.
- **RevisionAgent**: regole di formato esplicite (testo semplice, no Markdown, no emoji, MAIUSCOLE per enfasi), preservazione URL e date, lunghezza target (1-3 frasi per risposte semplici, max 10-15 punti per risposte articolate).

7.4.6 Timing nel Logger senza Overhead

I tempi di esecuzione sono misurati con `time.time()` (chiamata a basso costo) all'inizio e alla fine di ogni nodo del grafo. Il metodo `_format_timing_section()` del `PipelineLogger` traduce i nomi tecnici degli step in etichette leggibili tramite una mappa `step_labels` e formatta ogni tempo con 3 decimali. L'overhead introdotto è trascurabile rispetto alla latenza delle chiamate LLM.

7.5 Analisi Dinamica

L'analisi dinamica è condotta tramite test API eseguiti con Postman, estendendo la collection dell'Iterazione 4. Le variabili utilizzate sono sempre le stesse della precedente iterazione.

ID	Endpoint	Met.	Descrizione	Atteso
IT5-01	`/api/auth/profile`	PUT	Aggiornamento profilo con dati validi	200 OK
IT5-02	`/api/agent/query`	POST	Query `date_esami` con utente autenticato	200 OK

Tabella 7.2: Test API – Iterazione 5

I risultati confermano il corretto funzionamento dei nuovi endpoint di modifica profilo. Il flusso `date_esami` seleziona correttamente il calendario per utenti autenticati.

Capitolo 8

Design in Piccolo

In questo capitolo si analizzano due algoritmi chiave estratti dal codice sorgente. Per ciascuno si presenta il contesto applicativo, lo pseudocodice e un'analisi formale della complessità temporale.

8.1 Algoritmo 1 — Esecuzione del Workflow come visita di un DAG

8.1.1 Contesto

La classe `OrchestratorAgent` costruisce un grafo diretto aciclico (DAG) per coordinare l'esecuzione sequenziale e condizionale di cinque agenti specializzati. Ogni nodo del grafo corrisponde a un agente (Classifier, Query, WebSearch/ExamExtract, Generator, Revision) e gli archi rappresentano le dipendenze nel flusso dati ed esecutivo. Il framework LangGraph compila questo grafo e lo esegue tramite un ordinamento topologico, garantendo che ogni nodo venga visitato solo dopo che tutti i suoi predecessori siano stati completati.

La struttura del grafo, estratta dal metodo `_build_workflow()`, è la seguente:

Arco (u → v)	**Tipo**
`START` → `classify`	incondizionato
`classify` → `query`	incondizionato
`query` → `web_search`	condizionato: `category` ≠ `date_esami`
`query` → `exam_extract`	condizionato: `category = date_esami`
`web_search` → `generate`	incondizionato
`exam_extract` → `generate`	incondizionato
`generate` → `revise`	incondizionato
`revise` → `END`	incondizionato

Il grafo ha $|V| = 6$ nodi (agenti) e $|E| = 7$ archi (di cui 2 mutuamente esclusivi), ma durante una singola esecuzione vengono attraversati esattamente 5 nodi e 4 archi (un solo ramo condizionale è attivo).

8.1.2 Pseudocodice

L'Algoritmo 1 descrive la procedura di esecuzione del workflow, modellata come una visita topologica di un DAG con routing condizionale.

Algoritmo 1 Esecuzione del workflow (visita topologica del DAG)

Input: Grafo $G = (V, E)$ con nodi-agente, funzione di routing ρ, stato iniziale S_0
Output: Stato finale S_f con la risposta elaborata

```
procedure ExecuteWorkflow(G, S_0, ρ)
    S ← S_0                                          ▷ Inizializza lo stato condiviso
    order ← TopologicalSort(G)                       ▷ Ordine topologico dei nodi
    visited ← ∅
    entry ← nodo sorgente di G                       ▷ classify
    queue ← [entry]                                  ▷ Coda dei nodi pronti
    while queue ≠ ∅ do
        v ← queue.Dequeue()
        if v ∈ visited then
            continue
        end if
        S ← Execute(v, S)                            ▷ Esegui l'agente v sullo stato
        visited ← visited ∪ {v}
        for all arco (v, w) ∈ E do                   ▷ Esamina i successori
            if l'arco (v, w) è condizionato then
                if ρ(v, S) = w then                  ▷ Il routing seleziona w
                    queue.Enqueue(w)
                end if
            else
                queue.Enqueue(w)
            end if
        end for
    end while
    return S
end procedure
```

8.1.3 Analisi di complessità temporale

Costo della visita del DAG. L'ordinamento topologico e la successiva visita di un DAG con $|V|$ nodi e $|E|$ archi hanno complessità nota:

$$T_{\text{visita}} = O(|V| + |E|)$$

Nel nostro caso $|V| = 6$ e $|E| = 7$, quindi il costo strutturale della visita è $O(1)$ (costante, poiché il grafo è statico e di dimensione fissa).

Costo di esecuzione dei singoli nodi. Sia $T(v_i)$ il tempo di esecuzione dell'agente al nodo v_i. In una singola esecuzione del workflow vengono attraversati esattamente $k = 5$ nodi (un solo ramo condizionale è attivo). Il costo totale è:

$$T_{\text{pipeline}} = \sum_{i=1}^{k} T(v_i)$$

Poiché i nodi sono eseguiti **sequenzialmente** (ogni nodo dipende dallo stato prodotto dal precedente), non vi è parallelismo e il costo è la somma dei singoli costi.

Costo complessivo. Combinando i due contributi:

$$T_{\text{totale}} = O(|V| + |E|) + \sum_{i=1}^{k} T(v_i) = O(1) + \sum_{i=1}^{k} T(v_i) = O\left(\sum_{i=1}^{k} T(v_i)\right)$$

La complessità è dunque dominata dalla somma dei tempi di esecuzione dei singoli agenti, non dalla struttura del grafo.

8.2 Algoritmo 2 — Ordinamento e selezione Top-K dei risultati web

8.2.1 Contesto

Il metodo `WebAgent.search()` del modulo `web_agent.py` esegue una ricerca web tramite l'API Tavily, che restituisce fino a $n = 30$ risultati. Ciascun risultato possiede un campo `score` $\in [0, 1]$ che ne indica la rilevanza. L'algoritmo deve selezionare i $k = 5$ risultati con punteggio più alto per passarli al `GeneratorAgent`.

Nel codice sorgente, questa operazione è implementata come segue:

```
sorted_results = sorted(results,
                        key=lambda x: x.get("score", 0),
                        reverse=True)
top_results = sorted_results[:5]
```

Si tratta di un ordinamento completo seguito da una selezione per troncamento.

8.2.2 Pseudocodice

L'Algoritmo 2 descrive la procedura di selezione Top-K implementata nel codice tramite ordinamento.

Algoritmo 2 Selezione Top-K per ordinamento (approccio nel codice)

Input: Array $R[1 \dots n]$ di risultati, ciascuno con campo *score*; intero $k \leq n$
Output: Array $T[1 \dots k]$ contenente i k risultati con score più alto, ordinati per score decrescente

1: **procedure** TopK-Sort(R, n, k)
2: $\quad R' \leftarrow$ Sort$(R, \text{key} = \textit{score}, \text{ordine} = \text{decrescente})$ ▷ Ordinamento completo
3: $\quad T \leftarrow R'[1 \dots k]$ ▷ Selezione dei primi k
4: $\quad$ **return** T
5: **end procedure**

8.2.3 Analisi di complessità temporale

Python utilizza **Timsort**, un algoritmo di ordinamento ibrido (merge sort + insertion sort) con le seguenti complessità:

	Caso ottimo	**Caso medio**	**Caso pessimo**
SORT	$O(n)$	$O(n \log n)$	$O(n \log n)$
SLICE $[1 \dots k]$	$O(k)$	$O(k)$	$O(k)$
Totale	$O(n)$	$O(n \log n)$	$O(n \log n)$

La chiamata `sorted(results, key=..., reverse=True)` effettua un ordinamento completo di tutti gli n risultati. Il successivo slicing `[:5]` ha costo $O(k)$, trascurabile rispetto all'ordinamento. Dunque:

$$T_{\text{sort}}(n, k) = O(n \log n) + O(k) = O(n \log n)$$

Capitolo 9

Analisi Statica

L'analisi statica del codice è una tecnica fondamentale per valutare la qualità del software senza eseguirlo. In questo capitolo vengono presentati i risultati ottenuti dall'applicazione di due strumenti di analisi statica sul backend Python del progetto: **Radon** e **SonarQube for IDE**.

Il backend analizzato è composto da 20 file Python organizzati in 5 package (`agents/`, `auth/`, `logger/`, `models/`, e i moduli radice).

9.1 Strumenti Utilizzati

9.1.1 Radon

Radon è uno strumento open-source per l'analisi statica di codice Python che calcola diverse metriche di complessità e manutenibilità:

- **Raw Metrics**: conteggio di LOC, SLOC, righe di commento e righe vuote;
- **Cyclomatic Complexity (CC)**: misura il numero di cammini indipendenti attraverso il codice sorgente, secondo la formula di McCabe;

9.1.2 SonarQube for IDE

SonarQube for IDE è un'estensione di Visual Studio Code che esegue analisi statica in tempo reale rilevando:

- **Bug**: errori logici che possono causare comportamenti inattesi;
- **Code Smells**: pattern di codice che, pur funzionando, ne riducono la manutenibilità;
- **Vulnerabilità**: potenziali falle di sicurezza;
- **Security Hotspots**: aree di codice che richiedono revisione manuale per la sicurezza.

9.2 Metriche Raw (Radon)

Le metriche raw forniscono una panoramica quantitativa della dimensione e della struttura del codice sorgente. La Tabella 9.1 riporta i risultati per ciascun file significativo del progetto.

Tabella 9.1: Metriche Raw per file (Radon)

File	LOC	SLOC	LLOC
config.py	27	12	18%
db.py	18	10	8%
main.py	224	153	104%
agents/agent_state.py	75	48	62%
agents/classifier_agent.py	92	70	37%
agents/generator_agent.py	129	90	42%
agents/orchestrator_agent.py	437	307	188%
agents/query_agent.py	123	89	37%
agents/revision_agent.py	131	96	45%
agents/web_agent.py	418	299	221%
auth/controller.py	96	63	34%
auth/jwt_manager.py	56	31	30%
auth/profile_repository.py	54	29	35%
auth/service.py	113	67	49%
logger/pipeline_logger.py	343	241	221%
models/user.py	64	42	77%
TOTALE	**2.444**	**1.673**	**1.228**

Legenda delle metriche:

- **LOC**: Lines of Code, numero totale di righe nel file;
- **SLOC**: Source Lines of Code, righe contenenti codice effettivo;
- **LLOC**: Logical Lines of Code, istruzioni logiche (esclude righe vuote e commenti);

I file più corposi sono `orchestrator_agent.py` (437 LOC) e `web_agent.py` (418 LOC), coerentemente con il loro ruolo di moduli centrali della pipeline agentica.

9.3 Complessità Ciclomatica (Radon)

La complessità ciclomatica di McCabe misura il numero di cammini linearmente indipendenti attraverso il codice di una funzione o metodo. Valori più bassi indicano codice più semplice e facile da testare. Radon classifica la complessità con i seguenti gradi:

Tabella 9.2: Scala di valutazione della complessità ciclomatica (Radon)

Grado	Rischio	CC Range
A	Basso — codice semplice, nessun rischio	1–5
B	Basso — codice ben strutturato, rischio minimo	6–10
C	Moderato — codice mediamente complesso	11–15
D	Moderato — attenzione richiesta	16–20
E	Alto — codice complesso, rischioso	21–30
F	Molto alto — non testabile	31+

9.3.1 Risultati per Classe

La Tabella 9.3 riporta la complessità ciclomatica aggregata delle classi principali del progetto.

Tabella 9.3: Complessità Ciclomatica per classe

Modulo	**Classe**	**CC**	**Grado**
agents/web_agent.py	WebAgent	5	A
agents/generator_agent.py	GeneratorAgent	5	A
agents/classifier_agent.py	ClassifierAgent	4	A
logger/pipeline_logger.py	PipelineLogger	4	A
agents/query_agent.py	QueryAgent	4	A
agents/revision_agent.py	RevisionAgent	3	A
auth/jwt_manager.py	JWTManager	3	A
auth/service.py	AuthService	3	A
agents/orchestrator_agent.py	OrchestratorAgent	2	A
auth/controller.py	AuthController	2	A
auth/profile_repository.py	ProfileRepository	2	A
agents/agent_state.py	AgentState	1	A

Tutte le classi del progetto rientrano nel grado **A** (complessità bassa), con valori compresi tra 1 e 5.

9.4 Analisi SonarQube for IDE

L'analisi con SonarQube for IDE ha rilevato delle issue distribuite nelle seguenti categorie. Le issue sono state classificate per tipologia e severità.

9.4.1 Riepilogo delle Issue

Tabella 9.4: Riepilogo issue SonarQube per categoria

Categoria	N. Issue	Descrizione
Naming Convention	8	Nomi di metodi non conformi allo standard Python `snake_case`
Code Smell (codice commentato)	5	Blocchi di codice commentato da rimuovere
FastAPI Best Practice	4	Mancato uso di `Annotated` per la dependency injection
Documentazione HTTP	6	HTTPException non documentate nel parametro `responses`
Duplicazione Literal	2	Stringhe letterali duplicate (da estrarre in costanti)
Sicurezza	2	Password hardcoded in `db.py`; binding a `0.0.0.0`
Funzionalità async	2	Funzioni `async` senza operazioni asincrone
Parametri inutilizzati	2	Parametri di funzione non utilizzati
Cognitive Complexity	1	Metodo con Cognitive Complexity > 15
F-string vuota	1	Uso di f-string senza campi di sostituzione
API deprecata	1	Uso di `datetime.utcnow()` deprecato

9.4.2 Issue di Naming Convention

SonarQube ha rilevato che diversi metodi nei moduli `auth/` non rispettano la convenzione Python `snake_case`:

Tabella 9.5: Violazioni di naming convention

File	Metodo attuale	Nome suggerito
auth/jwt_manager.py	`generateToken`	`generate_token`
auth/jwt_manager.py	`validateToken`	`validate_token`
auth/jwt_manager.py	`validateFromRequest`	`validate_from_request`
auth/service.py	`createUser`	`create_user`
auth/service.py	`updateProfile`	`update_profile`
auth/service.py	`changePassword`	`change_password`
auth/profile_repository.py	`findById`	`find_by_id`
auth/profile_repository.py	`updateProfile`	`update_profile`

La convenzione PEP 8 raccomanda lo stile `snake_case` per nomi di funzioni e metodi in Python.

9.4.3 Issue sulla Cognitive Complexity

SonarQube ha rilevato che il metodo `WebAgent._select_calendar_link` in `web_agent.py` presenta una **Cognitive Complexity di 22**, superiore alla soglia raccomandata di 15.

La Cognitive Complexity si differenzia dalla Complessità Ciclomatica in quanto penalizza maggiormente le strutture di controllo annidate e i salti nel flusso logico. Questo risultato è coerente con il valore CC = 12 (grado C) riportato da Radon per lo stesso metodo.

Capitolo 10

Quality Evaluation basata su Test

Questo capitolo presenta i risultati della fase di valutazione da parte di esperti, in cui tre esperti di dominio hanno valutato indipendentemente la qualità delle risposte generate dagli agenti su 20 domande di studenti/ospiti rivolte al sistema. Tutte le 20 domande e le relative risposte sono documentate nel dataset di valutazione. Ogni esperto ha valutato ciascuna risposta lungo tre dimensioni di qualità, per un totale di 180 valutazioni individuali (3 esperti $\times$ 3 dimensioni $\times$ 20 domande).

Il processo di valutazione ha adottato un framework in cui ogni esperto ha compilato in modo indipendente una scheda di valutazione. Gli esperti hanno valutato ciascuna risposta dell'agente lungo tre dimensioni di qualità:

1. **Chiarezza e Completezza**: Quanto è chiara e comprensibile la risposta dell'agente? Questo include la chiarezza del linguaggio, la pertinenza rispetto alla domanda dello studente e la completezza informativa (ad esempio, dettagli, link di riferimento, passi d'azione consigliati).
2. **Deviazione dal Contesto**: La presenza di affermazioni irrilevanti o errate (allucinazioni), come informazioni false su UniBg, frasi prive di senso, contenuti fuori contesto rispetto alla domanda dello studente, ripetizioni eccessive o link inesistenti generati dall'agente.
3. **Punteggio (Complessivo)**: Il punteggio complessivo di qualità della risposta generata dall'agente per la domanda corrispondente. Tiene conto di tutti gli aspetti qualitativi: chiarezza, completezza, assenza di deviazioni e utilità pratica per lo studente.

Ogni dimensione è stata valutata su una scala Likert a 5 punti: (1) Insufficiente/Inaccettabile, (2) Sotto la Media/Sotto le Aspettative, (3) Nella Media/Soddisfa i Requisiti di Base, (4) Buono/Sopra le Aspettative e (5) Eccellente/Supera le Aspettative.

Le 20 domande coprono un'ampia gamma di argomenti universitari, fornendo una valutazione completa della capacità del sistema di gestire il recupero di informazioni, il ragionamento contestuale e la sintesi di dati da fonti multiple attraverso diverse categorie di domande.

10.1 Valutazione Complessiva della Qualità

La valutazione degli esperti ha rivelato risultati molto positivi, con una media complessiva di 4.64 (DS = 0.72) su tutte le 180 valutazioni. Ciò indica che le risposte generate dagli agenti soddisfano o superano costantemente le aspettative dal punto di vista degli esperti. La distribuzione delle valutazioni è fortemente sbilanciata verso i valori positivi, con

il 76.1% delle valutazioni classificate come "Eccellente" (5) e solo lo 0.6% (una singola valutazione) classificato come "Insufficiente" (1).

Metrica	**Valore**	**Conteggio**
Media Complessiva	4.64 ± 0.72	180
Valutazione 5 (Eccellente)	76.1%	137
Valutazione 4 (Buono)	14.4%	26
Valutazione 3 (Nella Media)	7.8%	14
Valutazione 2 (Sotto la Media)	1.1%	2
Valutazione 1 (Insufficiente)	0.6%	1

Tabella 10.1: Statistiche complessive della valutazione degli esperti. La distribuzione mostra prestazioni elevate con il 90.5% delle valutazioni Buone o Eccellenti, indicando una generazione di risposte di alta qualità da parte del sistema multi-agente.

Analizzando le singole dimensioni di qualità, si possono osservare pattern significativi lungo i tre assi di valutazione:

La **Deviazione dal Contesto** ha ottenuto il punteggio più alto con una media di 4.85 (DS = 0.40), indicando che le allucinazioni e le affermazioni fuori contesto sono molto rare nell'output del sistema. Un notevole 86.7% delle valutazioni per questa dimensione è risultato "Eccellente", e nessuna valutazione è rientrata nelle categorie "Sotto la Media" o "Insufficiente". Questo risultato valida l'efficacia dell'architettura a pipeline multi-agente, in cui l'agente di recupero web attinge informazioni dalle pagine ufficiali dell'università e l'agente di revisione effettua un controllo finale di qualità prima di fornire la risposta.

Dimensione di Qualità	**Media ± DS**	**Conteggio**	**Range**
Chiarezza e Completezza	4.53 ± 0.75	60	[3, 5]
Deviazione dal Contesto	4.85 ± 0.40	60	[3, 5]
Punteggio (Complessivo)	4.55 ± 0.89	60	[1, 5]

Tabella 10.2: Valutazione degli esperti per dimensione di qualità. Ogni dimensione ha ricevuto 60 valutazioni (3 esperti × 20 domande). La Deviazione dal Contesto raggiunge la media più alta, riflettendo allucinazioni minime. La dimensione Punteggio mostra il range più ampio e la variabilità più alta, indicando problemi occasionali di qualità significativi in risposte specifiche.

La **Chiarezza e Completezza** ha ottenuto 4.53 (DS = 0.75), dimostrando che il sistema produce costantemente risposte ben strutturate e informative. Tuttavia, il 15.0% delle valutazioni è risultato "Nella Media", indicando che alcune risposte mancavano di dettagli sufficienti o avrebbero potuto essere più chiare nel linguaggio e nei passi d'azione consigliati.

Il **Punteggio (Complessivo)** ha raggiunto una media di 4.55 (DS = 0.89) ma ha mostrato il range più ampio [1, 5] e la deviazione standard più alta. Questa dimensione cattura la valutazione olistica e include l'unica valutazione "Insufficiente" dell'intera valutazione. La dispersione più ampia riflette il fatto che, sebbene la maggior parte delle risposte sia eccellente nel complesso, alcune presentavano problemi di qualità significativi che hanno impattato la valutazione combinata.

Valutazione	Chiarezza e Completezza	Deviazione dal Contesto	Punteggio
5 (Eccellente)	68.3% (41)	86.7% (52)	73.3% (44)
4 (Buono)	16.7% (10)	11.7% (7)	15.0% (9)
3 (Nella Media)	15.0% (9)	1.7% (1)	6.7% (4)
2 (Sotto la Media)	0.0% (0)	0.0% (0)	3.3% (2)
1 (Insufficiente)	0.0% (0)	0.0% (0)	1.7% (1)

Tabella 10.3: Distribuzione delle valutazioni per dimensione di qualità. La Deviazione dal Contesto mostra la concentrazione più forte al livello Eccellente (86.7%), mentre la dimensione Punteggio presenta la distribuzione più ampia, riflettendo il suo ruolo di misura olistica della qualità.

10.2 Prestazioni per Domanda

Le prestazioni variano considerevolmente tra le 20 domande valutate. In particolare, 11 domande su 20 (55%) hanno ricevuto un punteggio medio perfetto di 5.00 su tutti gli esperti e le dimensioni, indicando prestazioni impeccabili su un'ampia gamma di tipologie di domande.

D#	Media ± DS	Posizione
D1	5.00 ± 0.00	1°
D2	5.00 ± 0.00	1°
D3	5.00 ± 0.00	1°
D9	5.00 ± 0.00	1°
D12	5.00 ± 0.00	1°
D13	5.00 ± 0.00	1°
D15	5.00 ± 0.00	1°
D16	5.00 ± 0.00	1°
D17	5.00 ± 0.00	1°
D18	5.00 ± 0.00	1°
D19	5.00 ± 0.00	1°
D5	4.89 ± 0.33	12°
D7	4.89 ± 0.33	12°
D6	4.78 ± 0.44	14°
D20	4.78 ± 0.44	14°
D4	4.00 ± 0.87	16°
D10	4.00 ± 0.50	16°
D14	4.00 ± 0.87	16°
D8	3.67 ± 0.71	19°
D11	2.89 ± 1.05	20°

Tabella 10.4: Valutazione degli esperti per domanda. Ogni domanda ha ricevuto 9 valutazioni (3 esperti × 3 dimensioni). Undici domande hanno ottenuto punteggi perfetti. D11 è un chiaro outlier con la media più bassa.

D11 si distingue come un chiaro outlier con una media complessiva di 2.89 (DS = 1.05). Analizzando il dettaglio per dimensione, la dimensione Punteggio per D11 ha raggiunto una media di solo 1.67 (con valutazioni individuali degli esperti di 2, 2 e 1), il punteggio per singola dimensione più basso dell'intera valutazione.

Posizione	Domanda	Media
Le 5 Domande con Prestazioni Migliori		
1	D1, D2, D3, D9, D12, D13, D15–D19 (11 domande)	5.00
2	D5	4.89
3	D7	4.89
4	D6	4.78
5	D20	4.78
Le 5 Domande con Prestazioni Peggiori		
1	D11	2.89
2	D8	3.67
3	D4	4.00
4	D10	4.00
5	D14	4.00

Tabella 10.5: Domande con le migliori e peggiori prestazioni. Ogni valutazione rappresenta la media su 3 esperti e 3 dimensioni di qualità (9 valutazioni individuali).

10.3 Analisi per Esperto

I tre esperti hanno dimostrato pattern di valutazione notevolmente coerenti, con medie individuali comprese tra 4.60 e 4.70.

Esperto	Media ± DS	Conteggio
Esperto 1	4.60 ± 0.74	60
Esperto 2	4.70 ± 0.65	60
Esperto 3	4.63 ± 0.78	60

Tabella 10.6: Valutazione degli esperti per valutatore. Ogni esperto ha fornito 60 valutazioni (3 dimensioni × 20 domande). Il range ristretto delle medie (4.60–4.70) indica che gli esperti hanno applicato standard di qualità coerenti.

Il range ridotto di 0.10 tra la media più alta e quella più bassa degli esperti indica che tutti e tre gli esperti hanno applicato standard di qualità simili. L'Esperto 2 è risultato leggermente più generoso in media (4.70), mentre l'Esperto 1 leggermente più conservativo (4.60), ma queste differenze sono trascurabili e ben all'interno della variazione attesa.

10.4 Inter-Expert Agreement

Per valutare la coerenza e l'affidabilità delle valutazioni degli esperti, è stato analizzato il livello di accordo tra i tre esperti indipendenti che hanno valutato tutti i 60 scenari (20 domande × 3 dimensioni). Questa analisi è fondamentale per comprendere se le valutazioni di qualità sono riproducibili e non meramente opinioni soggettive.

Le statistiche di accordo di base rivelano un pattern di valutazione altamente coerente tra gli esperti. Su 60 scenari valutati, gli esperti hanno assegnato la stessa valutazione in 44 casi (73.3%), rappresentando un accordo esatto. Consentendo una differenza di un punto sulla scala Likert, l'accordo ha raggiunto 60 casi (100.0%), indicando che gli esperti non hanno *mai* avuto un disaccordo superiore a un punto. La varianza media tra le tre valutazioni per ogni scenario è stata di 0.059 (± 0.099), confermando una dispersione molto bassa nei giudizi degli esperti.

10.4.1 Analisi del Kappa di Cohen

Per misurare l'accordo al di là di quanto ci si aspetterebbe per puro caso, è stato calcolato il coefficiente per ogni coppia di esperti. Il kappa di Cohen è una misura statistica che corregge per l'accordo che si verifica puramente per caso: un valore di 0 indica un accordo non migliore del caso, mentre 1.0 rappresenta un accordo perfetto. È stato utilizzato il *kappa ponderato quadratico* poiché la scala Likert è ordinale, il che significa che un disaccordo di 2 punti (ad esempio, valutazione 5 vs. 3) è da considerarsi peggiore di un disaccordo di 1 punto (ad esempio, 5 vs. 4).

I valori del kappa di Cohen a coppie tra i tre esperti sono risultati:

- **Esperto 1 vs Esperto 2**: $\kappa = 0.792$ (accordo sostanziale)
- **Esperto 1 vs Esperto 3**: $\kappa = 0.854$ (accordo quasi perfetto)
- **Esperto 2 vs Esperto 3**: $\kappa = 0.836$ (accordo quasi perfetto)

Il kappa di Cohen medio a coppie è risultato **0.827**, che secondo la scala interpretativa di Landis & Koch (1977) rientra nella categoria di *accordo quasi perfetto* (0.81–1.00). Si tratta di un risultato notevolmente forte rispetto a tipici studi di valutazione da parte di esperti, indicando che i tre esperti condividevano una comprensione molto coerente dei criteri di qualità e li hanno applicati in modo affidabile in tutti gli scenari.

10.4.2 Analisi del Kappa di Fleiss

Mentre il kappa di Cohen misura l'accordo tra coppie di valutatori, il *kappa di Fleiss* estende questo concetto a tre o più valutatori simultaneamente. Fornisce una misura unica dell'affidabilità inter-valutatore complessiva su tutti gli esperti. Per la nostra valutazione con tre esperti, il kappa di Fleiss è risultato **0.548**, che rientra nella categoria di *accordo moderato* (0.41–0.60) secondo la scala di Landis & Koch.

La differenza tra il kappa di Cohen medio a coppie (0.827) e il kappa di Fleiss (0.548) può essere spiegata da due fattori. In primo luogo, il kappa di Fleiss è intrinsecamente più conservativo poiché considera tutti e tre gli esperti simultaneamente. In secondo luogo, la distribuzione delle valutazioni è fortemente sbilanciata verso la categoria Eccellente (il 76.1% di tutte le valutazioni è 5), il che crea un alto accordo atteso per caso ($P_e = 0.606$), rispetto al quale viene confrontato l'accordo osservato ($\overline{P} = 0.822$). Questo effetto soffitto rende più difficile ottenere un kappa di Fleiss elevato anche con un forte accordo effettivo.

10.4.3 Interpretazione e Validità

Queste statistiche di affidabilità inter-valutatore dimostrano che il processo di valutazione degli esperti è stato altamente coerente. L'accordo quasi perfetto a coppie (kappa di Cohen = 0.827) indica che gli esperti hanno applicato i criteri di qualità con notevole coerenza. Il kappa di Fleiss moderato (0.548) riflette la difficoltà matematica di ottenere un alto accordo a tre vie quando la distribuzione delle valutazioni è fortemente concentrata, piuttosto che indicare un disaccordo genuino.

La percentuale perfetta di accordo ravvicinato (100% entro 1 punto) è particolarmente degna di nota: nessuna coppia di esperti ha mai avuto un disaccordo superiore a un singolo punto su alcuno scenario, il che rappresenta un livello eccezionale di consenso per

una valutazione qualitativa. Questo pattern conferma che il framework di valutazione e le dimensioni di qualità erano ben definiti e hanno consentito valutazioni affidabili e riproducibili.

10.5 Risultati Principali e Considerazioni

Dalla valutazione degli esperti emergono diverse conclusioni importanti:

1. **Alta qualità complessiva delle risposte degli agenti**: Il sistema ha raggiunto una valutazione media di 4.64/5.00 su tutte le 180 valutazioni, con il 90.5% delle valutazioni Buone o Eccellenti. Ciò dimostra che l'architettura multi-agente produce risposte costantemente di alta qualità per un'ampia gamma di domande universitarie.
2. **Allucinazioni minime**: La Deviazione dal Contesto ha ottenuto 4.85/5.00 con l'86.7% di valutazioni Eccellenti, validando l'efficacia dell'architettura a pipeline in cui l'agente web recupera informazioni da fonti ufficiali e l'agente di revisione effettua il controllo di qualità prima di fornire la risposta finale.
3. **Il 55% delle domande ha ricevuto punteggi perfetti**: Undici domande su 20 hanno raggiunto una media perfetta di 5.00 su tutti gli esperti e le dimensioni, dimostrando che il sistema eccelle nella gestione di una gamma diversificata di tipologie di domande.
4. **Alcune domande mostrano margini di miglioramento**: La domanda con il punteggio più basso (D11, media = 2.89) e alcune altre (D8, D4, D10, D14) hanno ottenuto punteggi inferiori alla media complessiva, evidenziando scenari specifici in cui la qualità delle risposte del sistema potrebbe essere migliorata.
5. **L'accordo tra esperti è eccezionalmente forte**: Con il 100% delle valutazioni entro un punto di differenza l'una dall'altra e un kappa di Cohen medio di 0.827 (accordo quasi perfetto), il framework di valutazione si è dimostrato altamente efficace nel catturare valutazioni di qualità coerenti. Questo livello di affidabilità inter-valutatore supera i benchmark tipici degli studi di valutazione da parte di esperti, conferendo forte credibilità ai risultati.

La valutazione degli esperti integra le metriche quantitative fornendo un giudizio umano su aspetti difficili da misurare algoritmicamente, come l'appropriatezza contestuale e l'utilità pratica per gli studenti. I risultati confermano che il sistema multi-agente AgenticUniBG è in grado di generare risposte di alta qualità per domande universitarie, identificando al contempo aree specifiche in cui un ulteriore perfezionamento della pipeline di recupero e generazione sarebbe benefico.

Capitolo 11

Conclusioni e Sviluppi Futuri

11.1 Riepilogo delle Iterazioni

Come emerso nei capitoli precedenti, il progetto è cresciuto in modo incrementale seguendo la logica AMDD: da una visione iniziale ad alto livello fino a una piattaforma multi-agente realmente utilizzabile per richieste universitarie reali. Ogni iterazione ha introdotto un incremento funzionale chiaro e verificabile, mantenendo continuità architetturale e tracciabilità delle decisioni.

L'**Iterazione 0** ha definito requisiti, user stories e direzione progettuale. Con l'**Iterazione 1** è stato costruito il primo MVP con pipeline base *classify* → *generate* → *revise*. L'**Iterazione 2** ha consolidato il sistema applicativo con autenticazione, profilo utente e integrazione database. L'**Iterazione 3** ha introdotto la gestione di sessione e memoria conversazionale, migliorando la continuità delle interazioni. L'**Iterazione 4** ha introdotto la ricerca web reale tramite Tavily, insieme a QueryAgent e WebAgent, aumentando in modo significativo la qualità informativa delle risposte. Infine, l'**Iterazione 5** ha completato l'evoluzione funzionale con routing condizionale per `date_esami`, gestione avanzata del profilo utente, timestamp temporale nei prompt e potenziamento della robustezza complessiva della pipeline.

Iterazione	**Contributo principale**
0 – Visione	Definizione requisiti, user stories e architettura target
1 – MVP	Pipeline base multi-agente e primo flusso end-to-end
2 – Auth	Registrazione/login, persistenza MongoDB e personalizzazione utente
3 – Sessione	JWT httpOnly cookie, verify/logout e memoria conversazionale
4 – Web Search	QueryAgent, WebAgent e recupero informazioni reali da fonti ufficiali
5 – Profilo + Routing Esami	Profilo utente completo, routing condizionale `date_esami`, timestamp e robustezza pipeline

Tabella 11.1: Sintesi evolutiva delle iterazioni

11.2 Obiettivi Raggiunti

Nel complesso, gli obiettivi principali del progetto possono considerarsi raggiunti. La piattaforma implementa un'architettura multi-agente modulare, capace di classificare le richieste, recuperare contenuti da fonti ufficiali e produrre risposte coerenti, contestualizzate e verificabili. L'integrazione tra orchestrazione, autenticazione e memoria conversazionale ha permesso di superare il comportamento stateless tipico dei prototipi iniziali.

Dal punto di vista applicativo, il sistema copre i casi d'uso centrali per studenti e utenti: accesso autenticato, personalizzazione della risposta in base al profilo, gestione della sessione e supporto a domande informative complesse. Sul piano ingegneristico, le scelte su logging, containerizzazione e separazione dei componenti hanno migliorato osservabilità, manutenibilità e riproducibilità del deployment.

I risultati della validazione (discussi nelle sezioni precedenti) confermano che la qualità complessiva delle risposte è elevata e che il comportamento della pipeline è stabile sui flussi principali già coperti dai test.

11.3 Sviluppi Futuri

Le prossime evoluzioni mirano a trasformare il sistema attuale, ora funzionalmente completo nelle sue componenti principali, in una soluzione ancora più robusta in produzione. Dopo il completamento dell'Iterazione 5, il focus si sposta sull'ottimizzazione qualitativa, sulla scalabilità e sull'operatività continua.

In parallelo, risultano strategici alcuni miglioramenti tecnici:

- introduzione di un Filter Agent per selezionare meglio i contenuti passati al generator;
- aggiunta di un Memory Agent per ottimizzare i tempi di risposta, salvando in un database le risposte;
- Supporto di lingue straniere, per esempio l'aggiunta di un profilo «Studente Erasmus» affinchè possa ricevere supporto nella sua lingua madre e informazioni precise relative ad un profilo di quel tipo;
- Feedback Agent per ottenere risposte migliori in base alla valutazione delle domande precedenti;

Infine, una direzione ad alto impatto è l'automazione dell'aggiornamento delle fonti istituzionali (in particolare calendari esami), così da mantenere nel tempo la qualità informativa senza aumentare il carico operativo manuale.